IOT DEVICE HEALTH AND SECURITY MONITORING PLATFORM WITH AI MAINTENANCE ASSISTANT

A PROJECT REPORT

Submitted by

Siddharth S (310624243179)

in partial fulfillment for the award of the Degree

of

BACHELOR OF TECHNOLOGY

In

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



EASWARI ENGINEERING COLLEGE,
(AN AUTONOMOUS INSTITUTION)
AFFILIATED TO ANNA UNIVERSITY,
CHENNAI

OCTOBER 2026

EASWARI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project report titled **"IOT DEVICE HEALTH AND SECURITY MONITORING PLATFORM WITH AI MAINTENANCE ASSISTANT"** is the Bonafide work of **"SIDDHARTH S (310624243179)",** who carried out the project work under my supervision.

HEAD OF THE DEPARTMENT
Dr. T. Kalai Selvi
Easwari Engineering College
Ramapuram, Chennai - 600089

SUPERVISOR
Ms. D S K Harshitha
Easwari Engineering College
Ramapuram, Chennai - 600089

EASWARI ENGINEERING COLLEGE

Certificate of Viva-voce-Examination

This is to certify that the Viva Voce Examination of the Project Work entitled “IoT Device Health and Security Monitoring Platform with AI Maintenance Assistant” submitted by Mr. Siddharth S (310624243179) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology was conducted on 22.10.2026 in the Department of Artificial Intelligence and Data Science, Easwari Engineering College, Ramapuram – Chennai.

Internal Examiner
Date:

ABSTRACT

Connected devices such as sensors, controllers, pumps and machine tools generate a continuous stream of readings, yet maintenance teams usually learn of a problem only after a fixed limit has been crossed, and the knowledge needed to interpret it stays in manuals that nobody has time to search. This project presents IoT Sentinel, a secure and locally deployable platform that monitors the health and security of heterogeneous IoT devices, predicts imminent limit breaches with an explanation, and answers maintenance questions from live data and uploaded documents with verifiable sources.

The platform accepts readings of eight signals over a REST API authenticated by per-device keys, applies alert limits that can be overridden for each machine type, detects silent devices through expiring heartbeat keys, and tracks firmware, security incidents, maintenance history and an audit trail. A logistic-regression model forecasts, for every signal, the probability that its limit will be crossed within the next twenty readings. Each prediction is explained by exact feature contributions, models can be retrained from telemetry, evaluated on newer data, versioned and rolled back, and a retrained model is activated only if it passes a validation gate. A knowledge base ingests PDF, DOCX, text and CSV files, and an AI Maintenance Assistant that runs on a local language model answers from read-only live-data tools and from documents retrieved by hybrid vector and keyword search under the user's own access rights. Citations are verified, document text is treated as untrusted data, and the assistant can never modify the system.

The system is implemented with React and TypeScript, Node.js and Express.js, MongoDB for operational data, PostgreSQL with pgvector for retrieval, Redis and Docker. It is verified by 176 backend tests, 39 front-end tests and a thirteen-step browser test, all passing. The prediction model reaches an AUC of 0.94 on held-out synthetic episodes; this describes the synthetic generator and not real machines, and the report states this and the other limitations of the work openly.

ACKNOWLEDGEMENT

We express our sincere gratitude to the Principal **Dr. P. Deiva Sundari** and the Management of our college for providing us with the opportunity to carry out this project work.

We convey our heartfelt thanks to the Head of the Department **Dr. T. KalaiSelvi**, Department of Artificial Intelligence and Data Science, for the continuous encouragement and support.

We are extremely thankful to our Project Guide **Ms. D. S. K. Harshitha**, Department of Artificial Intelligence and Data Science for valuable guidance, suggestions, and motivation throughout the completion of this project.

We would also like to extend our gratitude to all the faculty members of the Department of Artificial Intelligence and Data Science for their support.

Finally, we thank our parents and friends for their constant encouragement and help during the course of this project.

LIST OF TABLES

TABLE NO.	TITLE	PAGE No.
2.1	Existing approaches compared with the proposed system	8
3.1	Functional requirements	10
3.2	Non-functional requirements	10
3.3	Software requirements	11
3.4	Hardware requirements	12
5.1	Summary of the REST API	22
5.2	Evaluation of the built-in model on synthetic episodes	23
5.3	Results of the automated tests (run of 19 October 2026)	25

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE No.
4.1	System architecture of IoT Sentinel	14
4.2	Class diagram - devices, telemetry, alerts, rules and maintenance (MongoDB)	17
4.3	Class diagram - prediction, model versions, firmware, incidents and webhooks (MongoDB)	18
4.4	Class diagram - knowledge base and assistant (PostgreSQL + pgvector)	18
4.5	Sequence diagram - telemetry ingestion, alerting and prediction	19
4.6	Sequence diagram - the AI Maintenance Assistant answering a question	20
1	Sign-in screen	44
2	Fleet overview dashboard with live health, alerts and predicted risks	44
3	Device inventory with status, firmware and latest readings	45
4	Device detail page with live readings and forecast charts	45
5	Alerts screen (threshold, heartbeat and predictive alerts)	46
6	Anomaly prediction: fleet risk ranking and forecast chart with confidence band	46
7	Alert rules with per-machine-type limits	47
8	Firmware compliance and roll-outs	47
9	Security incident register	48
10	Knowledge base: upload and search	48
11	AI Maintenance Assistant answering from live data, with a citation and the tool used	49
12	AI Maintenance Assistant answering from an uploaded manual, with the source shown	49
13	Reports: preview with filters and CSV / PDF download	50
14	Audit log (administrators only)	50
15	User management: the two roles, administrator and viewer (administrators only)	51
16	Why this risk? Explanation of a prediction for one device	51

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE No.
	ABSTRACT	i
	ACKNOWLEDGEMENT	ii
	LIST OF TABLES	iii
	LIST OF FIGURES	iv
	TABLE OF CONTENTS	v
1	**INTRODUCTION**	1
	1.1 OVERVIEW	1
	1.2 PROBLEM DEFINITION	2
	1.3 OBJECTIVE	2
	1.4 SCOPE OF THE PROJECT	3
	1.5 MOTIVATION	4
2	**LITERATURE SURVEY**	5
	2.1 LITERATURE REVIEW	5
	2.2 EXISTING SYSTEM	7
	2.3 PROPOSED SYSTEM	7
3	**SYSTEM ANALYSIS**	9
	3.1 SYSTEM REQUIREMENTS	9
	3.2 SOFTWARE AND HARDWARE REQUIREMENTS	11
4	**METHODOLOGY**	13
	4.1 SYSTEM ARCHITECTURE	13
	4.2 MODULE DESCRIPTION	15
	4.4 CLASS DIAGRAM	17
	4.5 SEQUENCE DIAGRAM	19
5	**RESULTS AND DISCUSSION**	21
	5.1 IMPLEMENTATION OVERVIEW	21
	5.2 KEY FUNCTIONS (CODE EXPLANATION)	25
6	**CONCLUSION & FUTURE WORK**	31
	6.1 CONCLUSION	31
	6.2 FUTURE ENHANCEMENTS	32
	APPENDIX	33
	SCREENSHOTS OF THE PROJECT	44
	REFERENCES	52

CHAPTER 1

INTRODUCTION

This chapter deals with the background and the purpose of the project. It introduces the IoT Device Health and Security Monitoring Platform (IoT Sentinel), explains the problem that motivated it, states the objectives that guided the work, describes what is and is not covered, and explains why the work was worth undertaking.

1.1 OVERVIEW

The Internet of Things (IoT) connects sensors, controllers, cameras, meters and industrial machines to networks so that their condition can be observed and acted upon remotely. As the number of connected devices in factories, utilities and buildings grows, the people who look after them must watch an ever larger stream of readings, decide which device needs attention first, keep firmware up to date, react to security events and, at the same time, remember what the maintenance manuals say about each type of machine. Doing this with separate tools, spreadsheets and printed documents is slow and error prone.

IoT Sentinel is a web-based platform that brings these tasks together. Devices (or the built-in simulator) send readings such as temperature, CPU load, memory use, battery level, signal strength, vibration, pressure and motor current to a REST API. The platform stores the readings, compares them with limits that can be set for each machine type, raises alerts when a limit is crossed or a device stops reporting, and uses a small machine-learning model to estimate, before a limit is crossed, the probability that it is about to be crossed. Every prediction is accompanied by an explanation that shows how much each input contributed to the result.

In addition to monitoring, the platform contains a **Knowledge Base** and an **AI Maintenance Assistant**. Maintenance staff can upload manuals, procedures and inspection reports (PDF, DOCX, TXT, Markdown or CSV). The assistant answers questions in natural language by combining two sources: live data read from the platform through a fixed set of read-only tools, and passages retrieved from the documents the user is allowed to read. Each answer carries verifiable citations, and the assistant never changes any data. A language model that runs locally on the same machine (through Ollama) is used, so no document or reading leaves the installation.

The system is built with React and TypeScript for the user interface, Node.js with Express.js (TypeScript) for the REST API, MongoDB for operational data (users, devices, telemetry, alerts, rules and history), PostgreSQL with the pgvector extension for the retrieval-augmented generation (RAG) part, Redis for fast shared state and live updates, and Docker for packaging. It provides reports in CSV and PDF form, an audit log of security-relevant actions, and a complete automated test suite.

1.2 PROBLEM DEFINITION

Organisations that operate connected equipment usually face four related difficulties. First, monitoring is **reactive**: most dashboards only alert after a fixed threshold has already been crossed, which leaves little time to act. Second, **context is scattered**: readings are in one tool, firmware and security information in another, maintenance records in a spreadsheet and the knowledge needed to interpret a symptom in manuals that nobody has time to search. Third, the output of automated predictions is often a bare score, and a score that cannot be explained is difficult for a maintenance engineer to trust or to challenge. Fourth, adding a general-purpose AI chatbot to such a system creates new risks: the chatbot may invent facts, may reveal documents that belong to another user, and may follow instructions that are hidden inside an uploaded file.

Devices of different kinds also behave differently. A pump station and a smart sensor do not share the same normal vibration level or temperature limit, so a single global rule produces either too many false alarms or too few warnings. Finally, many real installations cannot send their data to an external cloud service, either because of cost or because of confidentiality, which rules out hosted AI services.

Problem statement: to design and implement a secure, explainable and locally deployable platform that monitors the health and security of heterogeneous IoT devices, predicts imminent limit breaches before they occur with an explanation for every prediction, and lets authorised users query live data and their own maintenance documents in natural language, with every answer traceable to its sources and without any ability to modify the monitored system.

1.3 OBJECTIVE

The main objective of the project is to build an end-to-end platform that turns raw device readings and maintenance documents into timely, explainable and trustworthy

decisions for maintenance staff. This main objective is divided into the following specific objectives.

The first group of objectives concerns monitoring. The platform must register devices of several machine types with their own signals and limits, accept readings over a REST API authenticated by a per-device key, detect threshold breaches and silent devices, keep a history of alerts, maintenance work and firmware updates, and push every change to open dashboards within a second.

The second group concerns prediction. The platform must forecast, for every signal of every device, the probability that its alert limit will be crossed within the next twenty readings; it must raise a predictive alert when that probability is high; it must show exactly why the model produced its value; it must allow a model to be retrained from the telemetry of one machine type, evaluate it on data it has not seen, keep every version, and refuse to activate a model that is not good enough.

The third group concerns knowledge and the AI assistant. The platform must ingest documents safely, split them into passages, index them for meaning-based and keyword search, and answer questions only from sources that the signed-in user may access. Answers must cite the passages or the live data they are based on, must say when nothing relevant was found, and must treat the text of uploaded documents as data rather than as instructions.

The last group concerns quality. The platform must protect accounts and device keys, record an audit trail without storing secrets, produce reports in CSV and PDF form, be packaged so that it starts with a single command, and be verified by automated unit, integration and end-to-end tests whose results are reported honestly, including their limits.

1.4 SCOPE OF THE PROJECT

The project covers the complete software stack of a monitoring platform: a responsive web interface with about twenty screens, a REST API with authentication and two user roles (administrator and viewer), a time-series style telemetry store with automatic expiry, a rule engine with limits that can be overridden for each machine type, heartbeat-based offline detection, a logistic-regression anomaly predictor with versioned retraining and explanations, firmware tracking with simulated roll-outs, a security incident register, signed webhook notifications, an audit log, eight kinds of reports, a document knowledge base and the AI Maintenance Assistant.

The platform can be used with real devices that can send an HTTP request, and it also contains a simulator that creates eighteen devices of eight machine types with realistic behaviour and faults, so that the whole system can be demonstrated without hardware. The project has been designed to run on a single computer with Docker, with an optional local language model.

Several things are deliberately outside the scope of this work. The prediction models predict that a **configured threshold will be crossed**; they do not predict equipment failure, because the platform has no failure labels. The model supplied with the system was trained on synthetic data, so its published accuracy describes the synthetic generator and not real machines. The platform does not terminate HTTPS itself (a reverse proxy should do this in production), does not send e-mail or SMS, does not perform optical character recognition on scanned documents, does not implement device protocols such as MQTT or CoAP, and is a single-organisation (not multi-tenant) system. These limits are discussed again in Chapter 6.

1.5 MOTIVATION

Unplanned downtime of pumps, machine tools and network equipment is expensive, and a large part of it could be avoided if a developing problem were noticed early. The readings that reveal such a problem, for example a slowly rising motor current or a growing vibration, are already being collected by most installations; what is often missing is a tool that watches them continuously, understands that different machines have different limits, and tells the engineer which machine to inspect first and why.

The second motivation is the recent progress of language models. They make it possible to ask a question such as "which machines need attention today and what does the manual say about their bearings?" and to receive an answer in seconds. However, an answer that cannot be checked is dangerous in a maintenance setting. The project therefore looks for a design in which the language model is only the last step: the facts come from fixed, read-only database queries and from retrieved passages, the sources are shown, the model cannot call anything except what the planner selected, and the system can refuse to answer.

Finally, the project is a practical opportunity to apply, in one coherent system, the topics studied during the programme: web development with a modern front-end framework, REST API design, relational and document databases, caching and messaging, authentication and secure coding, machine learning with a proper evaluation method, containerised deployment and software testing.

CHAPTER 2

LITERATURE SURVEY

This chapter deals with the existing knowledge and practice on which the project is built. It reviews the research and standards on IoT monitoring, predictive maintenance, anomaly detection, explainable models, web security and retrieval-augmented generation, describes how monitoring and maintenance support are done in existing systems and where they fall short, and then presents the proposed system that addresses those shortcomings.

2.1 LITERATURE REVIEW

Connected monitoring and Industry 4.0. Lee, Bagheri and Kao [1] describe a five-level architecture for cyber-physical systems in Industry 4.0 (connection, conversion, cyber, cognition and configuration). The lowest level collects data from machines, the middle levels turn the data into information and build a digital model of every machine, and the top levels support human decisions and self-adjustment. IoT Sentinel follows the same layering at a smaller scale: the REST ingest API is the connection level, the rule engine and the predictor are the conversion and cyber levels, and the dashboard, the explanations and the assistant provide the cognition level in which a person takes the decision. The configuration level, which would let the system change a machine by itself, is intentionally left out so that the assistant stays read-only.

Predictive maintenance. Mobley [2] explains that maintenance has moved from repairing machines after a failure (reactive), through servicing at fixed intervals (preventive), to servicing according to the measured condition of the machine (predictive). Condition measurements such as vibration, temperature, pressure and motor current are the basis of the predictive approach. The project uses exactly these signals for its industrial machine types, and it forecasts them so that the engineer is warned while there is still time to plan the work.

Anomaly detection. Chandola, Banerjee and Kumar [3] survey anomaly detection techniques and distinguish point anomalies, contextual anomalies and collective anomalies; a reading that is normal for one machine may be abnormal for another, which is a contextual anomaly. Ahmed, Mahmood and Hu [4] review anomaly detection techniques for network data, which is the setting of the security part of the platform. These surveys support two design decisions. First, limits and models are kept **per machine type**, so that the context is

part of the decision. Second, a simple and transparent classifier is preferred to a complex one when the amount of labelled data is small.

Explainable prediction. Logistic regression is described in detail by Hosmer, Lemeshow and Sturdivant [5]. Because the model is linear in its features, the contribution of every feature to the log-odds of the prediction is exactly the product of its weight and its standardised value. IoT Sentinel uses this property to show an exact explanation for every prediction instead of an approximate one, and it uses a time-ordered train-and-test split so that the reported evaluation reflects how the model will be used on future data.

Web architecture and authentication. Fielding [6] defines the REST architectural style (stateless requests, uniform resources and standard methods), which is the style of the platform's API. JSON Web Tokens are specified in RFC 7519 [7] and are used for the session of a signed-in user, while devices use a separate secret key that is stored only as a hash. The OWASP list of the ten most critical risks for applications built on large language models [8] was used as a checklist for the assistant, in particular the risks of prompt injection, insecure handling of model output, excessive agency and sensitive information disclosure.

Retrieval-augmented generation. Lewis and co-authors [9] show that a generator that is conditioned on passages retrieved from an external collection gives more factual answers to knowledge-intensive questions than a generator that relies only on what is stored in its parameters, and that the retrieved passages provide a form of provenance. The project applies this idea to maintenance documents. For the retrieval step, the Hierarchical Navigable Small World graph of Malkov and Yashunin [10] is the index that pgvector offers for approximate nearest-neighbour search, and reciprocal rank fusion, proposed by Cormack, Clarke and Büttcher [11], is used to merge the ranking obtained from embeddings with the ranking obtained from keyword search. The language model family used by default (Qwen2.5) is described in its technical report [12] and is based on the transformer architecture of Vaswani and co-authors [13].

Security of language-model applications. Greshake and co-authors [14] demonstrate that instructions hidden in data that an application retrieves (so-called indirect prompt injection) can take control of an LLM-integrated application. This is the reason why, in IoT Sentinel, the text of documents is never placed in the system prompt, is wrapped in explicit delimiters, has its look-alike markers and citations neutralised, and why the planner that chooses the tools is a deterministic program and not the model. The same study motivates the rule that the assistant has no tool that can write.

Taken together, the literature supports the central design of the project: monitor the condition of machines with context-specific limits, forecast with a model whose reasoning can be inspected, ground every generated answer in retrieved or measured evidence, and treat both user input and retrieved text as untrusted.

2.2 EXISTING SYSTEM

In many small and medium installations, device monitoring is still done by a combination of manual rounds, vendor-specific tools for each class of equipment and spreadsheets in which readings, inspection dates and repairs are recorded. Maintenance procedures are kept in printed manuals or in shared folders, and the engineer searches them by hand when a problem appears.

Larger installations use general IoT and cloud platforms (for example the device-management services offered by the major cloud providers, or open-source dashboards such as ThingsBoard). These platforms are good at collecting telemetry and drawing charts, and they usually offer threshold rules. Predictive features, explanations, document search and conversational assistance, where available, are separate paid services that require sending data to an external provider.

The existing approaches therefore share the following limitations. Alerts are raised only after a fixed limit has been crossed. A single limit is often applied to devices that do not behave alike. Predictions, when they exist, are not explained. Operational data, security events, firmware status and maintenance knowledge live in different places, so the engineer must assemble the picture by hand. General-purpose chatbots that are added on top of such systems are not grounded in the organisation's own documents and live data, can reveal information to users who should not see it, and can be steered by text hidden in files. Finally, solutions that depend on an external cloud are unsuitable where data must stay on the premises.

2.3 PROPOSED SYSTEM

The proposed system, IoT Sentinel, is a single platform that removes these limitations. It monitors devices through limits that are defined per machine type, detects both threshold breaches and silent devices, and adds a predictive layer that warns before a limit is crossed. Each prediction is explained with exact feature contributions, each trained model is

versioned and can be rolled back, and a retrained model is activated only when it passes a validation gate on data it has not seen.

Operational data, firmware compliance, security incidents, maintenance history and the audit trail are kept in one system. Maintenance documents are added to a private-by-default knowledge base and are searched with a combination of meaning-based and keyword search. The AI Maintenance Assistant answers questions from live data and from those documents, always shows its sources, refuses to guess when nothing relevant is found, can never modify the system, and runs entirely on the local machine. Reports in CSV and PDF form are available for management and for audits.

Aspect	**Existing approaches**	**Proposed system (IoT Sentinel)**
Alerting	Fixed threshold, after the event	Per-type thresholds plus a prediction before the event
Explanation of predictions	Rarely available, or a bare score	Exact per-feature contribution, model version and data source shown
Model management	Replaced without evaluation	Versioned, time-ordered evaluation, validation gate, rollback
Maintenance knowledge	Separate manuals and folders	Uploaded to a searchable, access-controlled knowledge base
Question answering	None, or an ungrounded chatbot	Grounded in live data and documents, with verifiable citations
Data privacy	Often sent to an external service	Models and data stay on the local machine
Chatbot safety	No defence against hidden instructions	Sources treated as data, read-only tools, deterministic planner
Traceability	Limited logs	Audit log of security-relevant actions without secrets
Reporting	Manual export	Eight report types as streamed CSV and PDF, simulated data marked

Table 2.1: Existing approaches compared with the proposed system

CHAPTER 3

SYSTEM ANALYSIS

This chapter deals with the analysis of what the system must do and what it needs in order to run. It lists the functional and non-functional requirements, identifies the users and the constraints under which the system works, and specifies the software and the hardware that were used to develop, test and run the platform.

3.1 SYSTEM REQUIREMENTS

Two kinds of users interact with the system. An **administrator** can change everything: register and deactivate devices, edit limits and machine types, start retraining, manage users, webhooks and shared documents, and read the audit log. A **viewer** can read dashboards, alerts, predictions, reports and shared documents, can upload private documents and use the assistant, but cannot change any monitored data. The first account created on a fresh installation becomes the administrator. A device is a third kind of actor: it is not a person, it authenticates with a secret key, and it can only send readings.

The functional requirements that were derived from the objectives are summarised in Table 3.1.

ID	Requirement
FR1	Users register and sign in; passwords are stored hashed; sessions expire; the first user becomes administrator; viewers are read-only on the server.
FR2	Administrators register devices of defined machine types, receive a device key once, rotate the key, deactivate a device without deleting its history, and delete a device with its data.
FR3	Devices send readings of up to eight signals (temperature, CPU, memory, battery, signal, vibration, pressure, motor current) and heartbeats; invalid values are rejected.
FR4	The system evaluates alert limits that apply to the machine type, opens, refreshes and automatically resolves threshold alerts, and raises a critical alert when a device stops reporting.
FR5	The system forecasts the probability of crossing each limit within the next 20 readings, raises predictive alerts, estimates time to breach, and stores prediction snapshots with their explanation.
FR6	Administrators retrain a model from the telemetry of a machine type; the result is evaluated on newer data, stored as a version, activated only if it passes the gate, and can be rolled back.
FR7	The system tracks firmware releases and compliance, performs simulated roll-outs, keeps a security incident register and per-device maintenance history.
FR8	Alerts, incidents and changes reach open browsers by push updates; critical events are sent to signed webhooks.

ID	Requirement
FR9	Users upload PDF, DOCX, TXT, Markdown and CSV documents, which are validated, indexed, searchable, downloadable and deletable; private by default, shareable by an administrator.
FR10	The AI assistant answers questions from live data and accessible documents, streams the answer, shows verifiable citations, keeps owner-only conversation history, and cannot change data.
FR11	Users preview reports and download them as CSV or PDF; simulated devices are marked.
FR12	Administrators read and export an audit log of sign-ins, administrative changes, document access, assistant tool use and report exports.
FR13	A built-in simulator creates demonstration devices and lets the operator pause the fleet or trigger a fault.

Table 3.1: Functional requirements

The non-functional requirements describe how well the system must perform its work and are summarised in Table 3.2.

Quality	Requirement
Security	Bcrypt password hashing; signed tokens with a 12-hour life; device keys stored only as SHA-256 hashes; role checks on the server; rate limits for the API, devices, logins, uploads, reports and assistant; protection against server-side request forgery in webhooks; CSV formula neutralisation; upload validation by file content.
Privacy	Documents private by default; the same access rule applied to listing, search, download and retrieval; conversations visible only to their owner; local language model, so that no data leaves the machine.
Reliability	Readings are stored before analysis; failures of the language model or of Redis degrade a feature but do not stop monitoring; uploads are queued and retried; models are never replaced by a worse one.
Performance	Dashboard updates by push within about a second; large exports are streamed in batches so memory does not grow with the number of rows; old telemetry expires automatically.
Honesty	Probabilities mean a threshold will be crossed, not that equipment will fail; models trained on synthetic data are labelled; simulated devices are marked in reports and in assistant answers.
Usability	Responsive interface with light and dark themes, clear empty, loading and error states, confirmation before destructive actions.
Maintainability	TypeScript throughout, versioned database migrations for the relational part, automatic index creation for the document part, automated tests and static checks, documentation of architecture and operation.
Portability	Complete stack started with Docker Compose; configuration only through environment variables; no secrets in the source code.

Table 3.2: Non-functional requirements

The system works under the following constraints and assumptions. The language model and the embedding model are small enough to run on a consumer computer; the default chat model needs about five gigabytes of graphics memory, and a smaller model can

be selected by configuration. Devices can reach the API over HTTP. Time is measured by the server clock. Model accuracy figures can only be interpreted for the data the model was trained on.

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

The software used to build and run the platform is listed in Table 3.3. All of it is free or open source.

Software	Version used	Purpose
Node.js	24.x (22 or later supported)	Runtime of the REST API and of the build tools
TypeScript	7.0	Typed language for the API and the front end
Express.js	5.2	REST API framework
React / React Router	19.3 / 7.18	User interface
Vite	8.3	Front-end build tool and development server
TanStack Query, Recharts	5.104, 3.10	Data fetching and caching; charts
MongoDB	7.0 (Node.js driver 7.7)	Operational data store
PostgreSQL with pgvector	16 with pgvector 0.8.1	Knowledge base, embeddings and conversations
Redis	7	Heartbeats, latest values, caches, rate limits, publish/subscribe
Ollama	0.33	Local runtime for the language and embedding models
Language models	qwen2.5:7b (default), phi4-mini; qwen3-embedding:0.6b	Answer generation; document and query embeddings (1024 dimensions)
Libraries	jsonwebtoken, bcryptjs, zod, helmet, multer, pdfjs-dist, mammoth, csv-parse, pdfkit	Authentication, validation, security headers, uploads, document parsing, PDF reports
Docker / Docker Compose	29.7 / 5.4	Packaging and start-up of the five services
Vitest, Supertest, Testing Library, Playwright	5.0, 7.3, 16.3, 1.64	Unit, API, component and end-to-end tests
Web browser	Google Chrome (current)	Using the application; end-to-end tests
Git and GitHub	-	Version control and continuous integration

Table 3.3: Software requirements

The hardware on which the system was developed and tested is given in Table 3.4 together with the minimum configuration that is sufficient for the monitoring features and the recommended configuration for the AI assistant.

Item	Development and test machine	Minimum (monitoring only)	Recommended (with assistant)
Processor	Laptop-class multi-core CPU	Dual-core, 2 GHz	4 or more cores
Memory	15.8 GB	4 GB	16 GB
Graphics	NVIDIA GeForce RTX 4050 Laptop GPU, 6 GB	Not required	GPU with 6 GB or more, or a smaller model on the CPU
Storage	SSD	5 GB free	15 GB free (models, images, data volumes)
Operating system	Windows 11 with Docker Desktop	Windows, Linux or macOS with Docker	Same
Network	Local	Local network to the devices	Same; HTTPS reverse proxy for production

Table 3.4: Hardware requirements

On the development machine the default chat model (qwen2.5:7b) did not fit into the six gigabytes of graphics memory together with the desktop, so the smaller phi4-mini model was used for the live tests and for the screenshots of this report. The model is selected by a single configuration variable.

CHAPTER 4

METHODOLOGY

This chapter deals with the method that was followed to design the system. It presents the architecture of the platform, describes every module and the responsibility it carries, and uses class and sequence diagrams to show the structure of the data and the order in which the components cooperate to handle a device reading and a question put to the assistant.

4.1 SYSTEM ARCHITECTURE

The platform follows a layered client-server architecture, shown in Figure 4.1. The user reaches the system through a web browser. The **presentation layer** is a single-page application written in React and TypeScript; it is served by Nginx, which also forwards every request that starts with /api to the back end and is configured so that streamed answers and large uploads pass unhindered. The application keeps its data fresh by combining periodic requests with a push channel based on Server-Sent Events (SSE), so that a new alert appears on an open screen without a refresh.

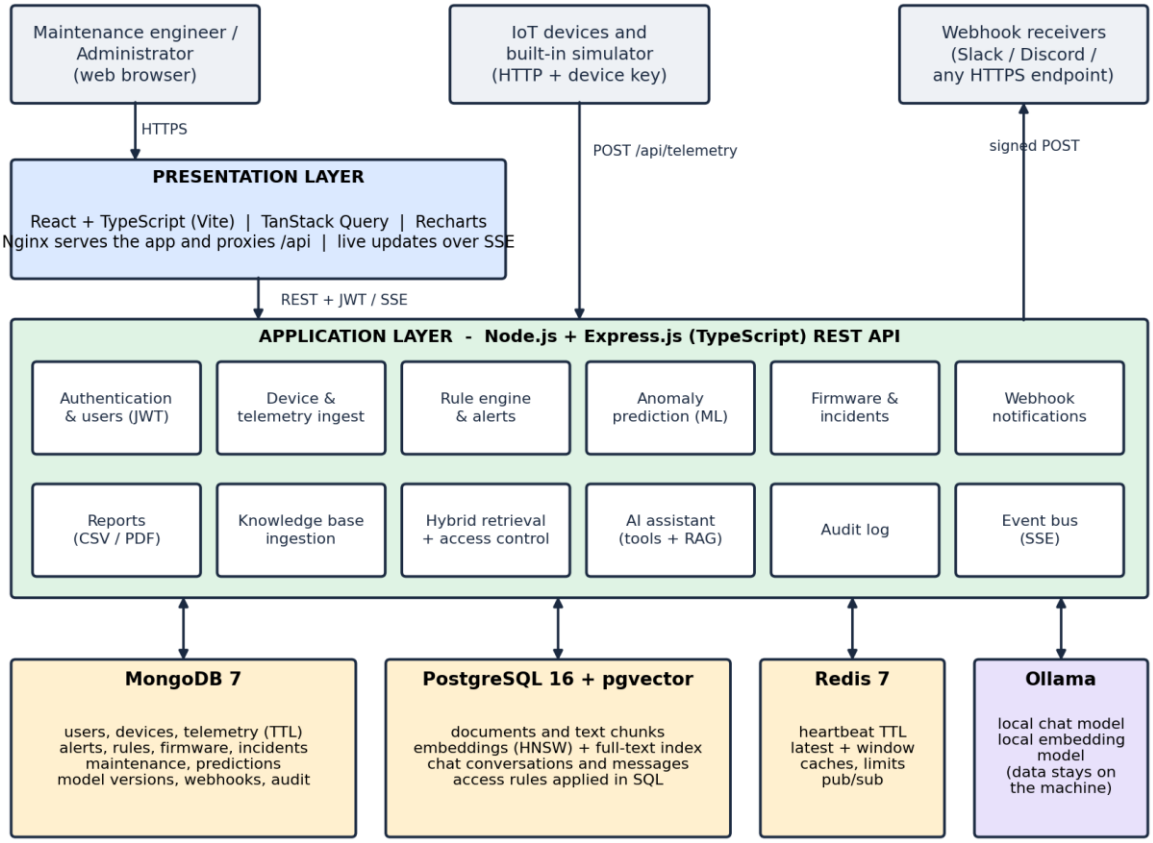


Figure 4.1: System architecture of IoT Sentinel

The **application layer** is a Node.js and Express.js REST API written in TypeScript. It is divided into modules (authentication, devices, telemetry, rules and alerts, prediction, firmware and incidents, webhooks, reports, knowledge base, assistant and audit) that share the same security middleware: security headers, a body-size limit, rate limiting and token authentication. Devices and simulated devices reach the API on a separate path that is authenticated by a device key and is not subject to the user-role rules.

The **data layer** uses three stores, each for the work it does best. MongoDB keeps the operational data, whose records are document-shaped and whose largest collection, telemetry, is time-stamped and expires automatically through a time-to-live index. PostgreSQL with the pgvector extension keeps the knowledge base, because retrieval needs vector similarity search, full-text search and an access rule evaluated in the same query. Redis keeps the data that must be fast and shared between processes: the heartbeat of each device with an expiry time, the latest reading and a rolling window of recent readings used by the predictor, short-lived response caches, rate-limit counters, a lock that allows one answer at a time per conversation, and the publish/subscribe channel behind the push updates.

Two external actors complete the picture. A local **Ollama** service provides the chat model and the embedding model; it is the only component that touches document text, and it never receives data from other users because retrieval has already filtered the passages. **Webhook receivers** (for example Slack or Discord) receive signed notifications for critical alerts, predicted breaches and serious incidents.

4.2 MODULE DESCRIPTION

The platform is made of the following modules. Each is a group of routes, services and screens with a single responsibility.

Authentication and user management. Registration, sign-in, profile and password change. Passwords are hashed with bcrypt, the session token is a JSON Web Token valid for twelve hours, and the user record is read again on every request so that role changes and deleted accounts apply immediately. The first user becomes the administrator; later users are viewers until promoted, and the last administrator cannot be removed. Failed sign-ins are limited per address.

Device and machine-type management. A machine type declares which of the eight signals it reports and the baselines used by the simulator. A device belongs to one type, has a human-friendly asset number, a location, a firmware version and a device key that is shown once and stored only as a hash. A device can be taken out of service without losing its history, which closes its alerts, stops its readings and removes it from the fleet counts.

Telemetry ingestion. The device endpoint validates the key and the ranges of the values, stores the reading in MongoDB, updates the hot state in Redis, evaluates the rules and the predictor, and updates the device status. A heartbeat endpoint reports liveness only. Old readings expire automatically after the retention period.

Rule engine and alerting. Default limits exist for each signal and can be overridden for each machine type. A reading beyond a limit opens a threshold alert, which is refreshed while the condition lasts and resolved automatically when the signal recovers. A sweeper checks the Redis heartbeat keys; when a key has expired the device is marked offline and a critical heartbeat alert is raised.

Anomaly prediction and model management. For every signal the module extracts five features from the last twenty readings and a logistic-regression model returns the probability that the limit is crossed within the next twenty readings. A predictive alert opens

above 50 percent. Models exist for the whole platform and, after retraining, for individual machine types. Each training run is stored as a version with its evaluation, activated only if it passes the validation gate, and can be rolled back. Every prediction can be explained.

Firmware and security incidents. Firmware releases are recorded for each machine type, compliance shows which devices run an old version or lack a security release, and a controlled roll-out (simulated in this project) advances in steps and updates the device version. Security incidents are reported with a category, a severity and a status workflow, and are summarised on the dashboard.

Maintenance history. Inspections, repairs, replacements, calibrations and notes are recorded for each device. Activation, deactivation and firmware updates add entries automatically, so the history of a machine is complete.

Notifications and live updates. Webhooks send signed JSON messages (HMAC-SHA256) for critical alerts, predicted breaches and high-severity incidents; destinations on private networks are refused and every delivery is logged. A Redis publish/subscribe channel feeds an SSE stream so that all open dashboards are refreshed immediately.

Reports. Eight reports (device health, telemetry history, alert history, predictive summary, machine-type comparison, maintenance history, model performance and knowledge-base inventory) can be previewed and downloaded as CSV, which is streamed in batches and protected against spreadsheet formula injection, or as a paginated PDF. Simulated data is marked and every export is audited.

Knowledge base. Uploaded files are validated by their extension and their content, queued, parsed (PDF page by page, DOCX, CSV rows as readable lines, text), cleaned, split into overlapping passages, converted into 1024-dimension embeddings and stored. A document is private to its owner unless an administrator shares it. Retrieval combines vector search and keyword search and applies the same access rule used everywhere else.

AI Maintenance Assistant. A deterministic planner chooses, from twelve fixed read-only tools, those that the question needs (fleet status, risk ranking, alerts summary, prediction explanation, maintenance history and so on). The tools and the retrieval step produce labelled sources, a prompt is built in which source text is delimited and neutralised, the language model streams its answer, and the citations are verified against the sources before the answer is stored. Conversations belong to their owner, can be renamed, exported and deleted, and the assistant cannot modify any record.

Audit log. Sign-ins, failed sign-ins, administrative changes, document uploads and downloads, assistant tool use and report exports are recorded with the user, the address and non-secret details. Passwords, keys, document contents and chat text are never written to it. Administrators can filter and export it.

Simulator. The simulator creates eighteen devices of eight machine types with seeded history, produces readings with realistic noise and slow drifts, and can pause the fleet or trigger a fault on a chosen device so that detection and prediction can be demonstrated on demand.

4.4 CLASS DIAGRAM

The persistent data of the platform is described by the classes of Figures 4.2, 4.3 and 4.4. Figure 4.2 shows the core monitoring data kept in MongoDB. A **DeviceType** declares the signals and carries the **AlertRules** that apply to it. A **Device** belongs to a type and owns its **Telemetry** (the readings, expired by a time-to-live index), its **Alerts** and its **MaintenanceRecords**.

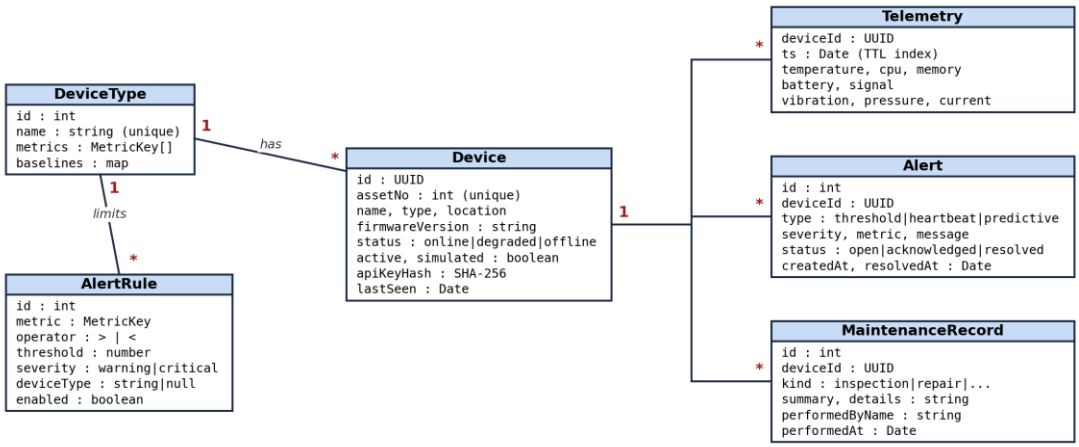


Figure 4.2: Class diagram - devices, telemetry, alerts, rules and maintenance (MongoDB)

Figure 4.3 shows the prediction and operations classes. A **PredictionSnapshot** records what the model estimated for one signal of one device, together with the model version and the full explanation, and refers to the **ModelVersion** that produced it. A **FirmwareUpdate** records a roll-out for a device, a **SecurityIncident** may or may not be tied to a device, and a **Webhook** notifies external receivers about incidents and alerts.

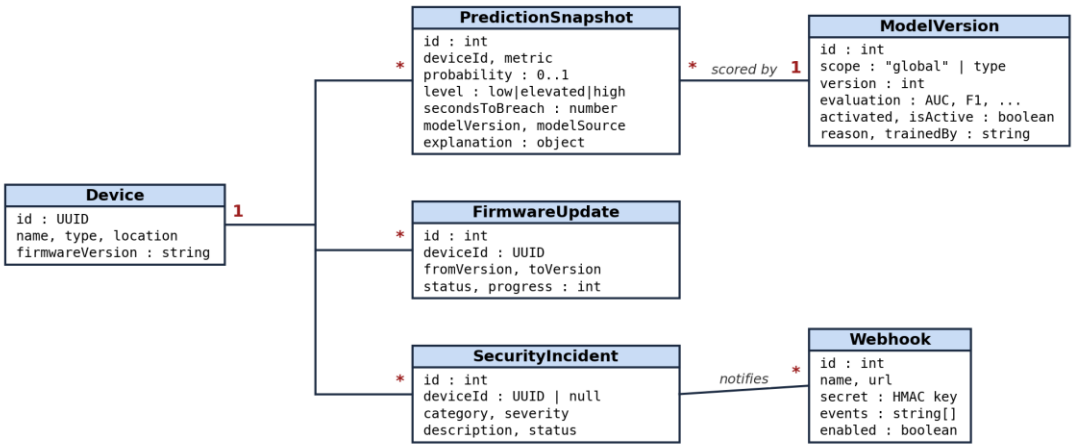


Figure 4.3: Class diagram - prediction, model versions, firmware, incidents and webhooks (MongoDB)

Figure 4.4 shows the knowledge and assistant classes, which live in PostgreSQL. A **User** (kept in MongoDB, referenced here only by its identifier) owns **KbDocuments**, each of which is split into **KbChunks** that hold the text, the page and section, the embedding vector and a full-text vector. A user also owns **Conversations**, each containing **Messages** that store the answer together with the sources and the tool calls it was based on. The **Tool** and **LLMProvider** classes show the two external capabilities that the assistant can use: the fixed read-only tools and the language model.

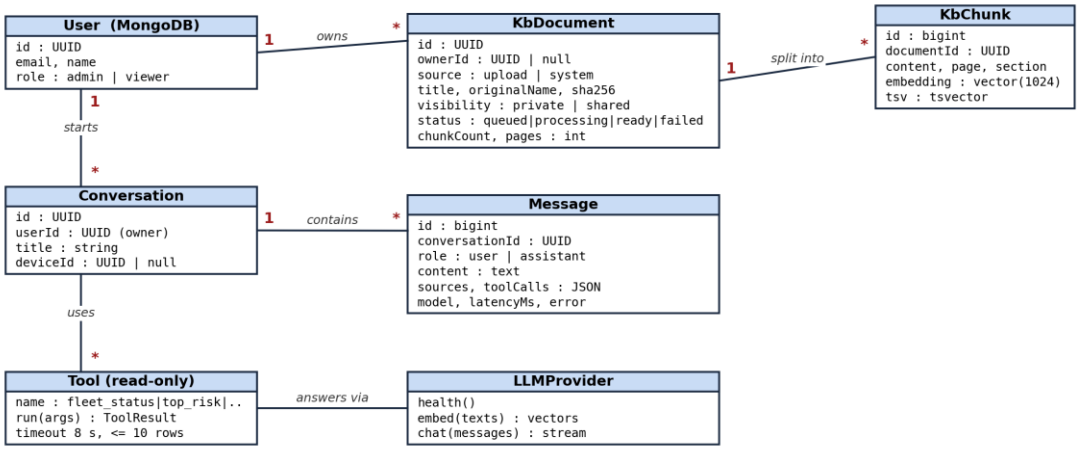


Figure 4.4: Class diagram - knowledge base and assistant (PostgreSQL + pgvector)

The references between the two databases are plain identifiers (UUID strings for users and devices) and not foreign keys, because a database cannot enforce a key that points into another database; the application checks and resolves them.

4.5 SEQUENCE DIAGRAM

Figure 4.5 shows what happens when a device sends a reading. The device posts the values with its key. The API looks up the device by the hash of the key and refuses a deactivated device, stores the reading, updates the latest value, the rolling window and the heartbeat expiry in Redis, evaluates the threshold rules and opens, refreshes or resolves an alert, asks the predictor to score the last twenty readings, stores a prediction snapshot with its explanation, publishes an event, and finally answers with status 202. Redis delivers the event to the open dashboards, which refresh the affected screens.

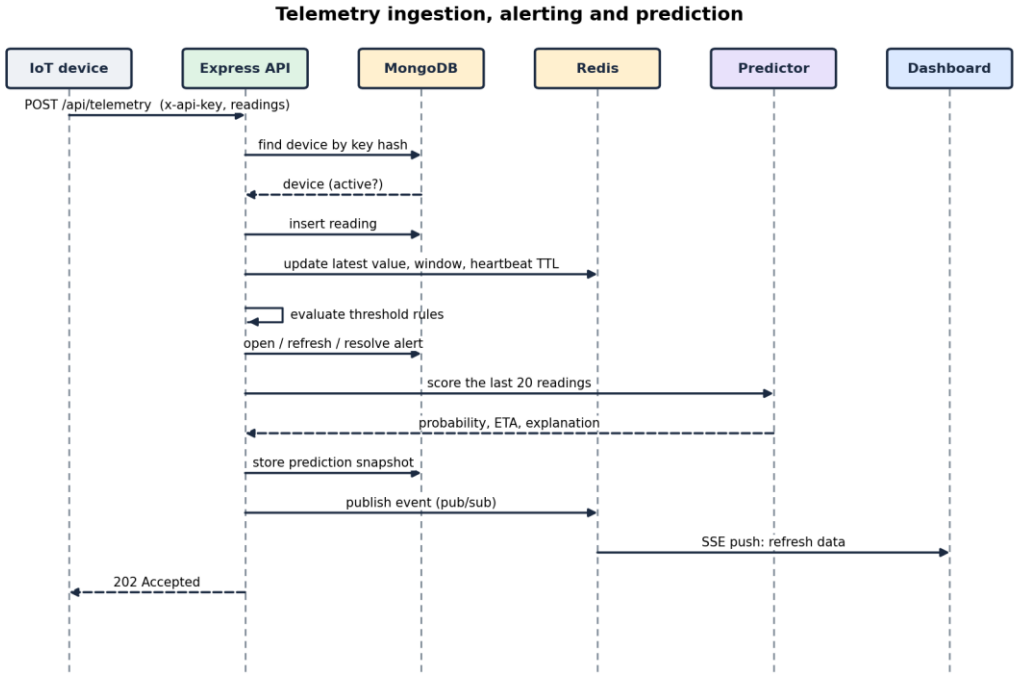


Figure 4.5: Sequence diagram - telemetry ingestion, alerting and prediction

Figure 4.6 shows how one question is answered. The browser sends the question with the user's token. The API saves it, lets the planner choose the read-only tools, runs them against MongoDB, embeds the question through Ollama and searches PostgreSQL with the owner-or-shared filter, builds the prompt in which every source is labelled and delimited, and streams the model's answer back as tokens. When the stream ends, the citations are checked against the sources that were supplied, the answer is saved with its sources and tool calls, and an audit entry that contains no message text is written.

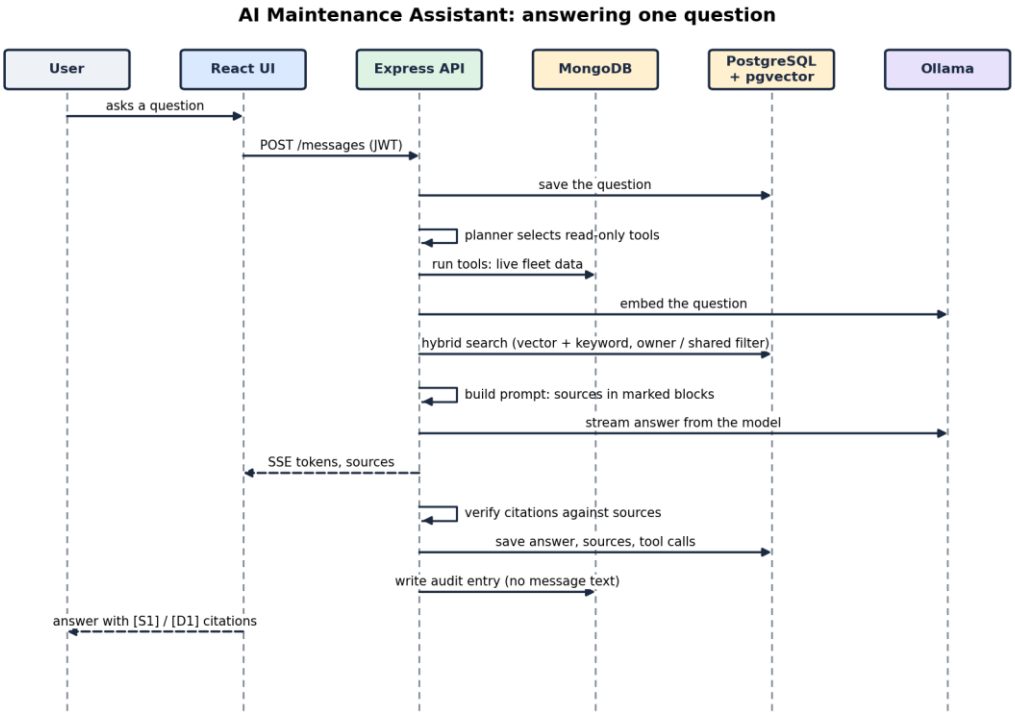


Figure 4.6: Sequence diagram - the AI Maintenance Assistant answering a question

CHAPTER 5

RESULTS AND DISCUSSION

This chapter deals with how the design was turned into a working system and what the finished system achieves. It describes the implementation, reports the results of the model evaluation and of the automated tests exactly as they were obtained together with their limits, and then explains, with code, the key functions on which the behaviour of the platform depends.

5.1 IMPLEMENTATION OVERVIEW

The system was implemented in TypeScript from the data layer to the user interface, in the following order: authentication and device management; telemetry ingestion with rules and alerts; the simulator; the predictor with its evaluation and explanations; firmware, incidents, webhooks and real-time updates; model versioning, maintenance history and the audit log; the knowledge base; the assistant; reports; and finally the tests, the container packaging and the documentation. The source is organised in three packages: the **backend** (routes, services, the machine-learning code and the database access), the **frontend** (pages, components and the API client) and the **end-to-end tests**. The complete structure is listed in the appendix.

The back end exposes the REST API summarised in Table 5.1. Every route validates its input with a schema, every change is authorised on the server according to the user's role, and errors are returned as a JSON message with a suitable status code (400 for invalid input, 401 when sign-in is needed, 403 when the role does not allow the action, 404, 409 for conflicts, 422 when there is not enough data and 429 when a rate limit is exceeded).

Area	Main endpoints	Access
Authentication	POST /auth/register, /auth/login; GET, PATCH /auth/me; POST /auth/me/password	Public (sign-in), user
Devices	GET, POST /devices; GET, PATCH, DELETE /devices/:id; POST /devices/:id/key; telemetry, readings, history, maintenance	User (read), administrator (write)
Device ingest	POST /telemetry, POST /telemetry/heartbeat (header x-api-key)	Device key
Machine types and rules	/device-types, /rules, PUT /rules/override	User (read), administrator (write)

Area	Main endpoints	Access
Alerts and incidents	GET /alerts, PATCH /alerts/:id; /incidents, /incidents/summary	User, administrator for changes
Prediction	/anomaly/predictions, /anomaly/explain/:id, /anomaly/history, /anomaly/models, POST /anomaly/retrain	User, administrator for training
Firmware	/firmware/releases, /firmware/compliance, /firmware/updates	User, administrator for roll-outs
Notifications and users	/webhooks, /users, /audit	Administrator
Knowledge base	/knowledge (upload, list, search, download, reindex, delete)	User (own and shared documents)
Assistant	/assistant/conversations, POST .../messages (streamed), /assistant/health	User (own conversations)
Reports	/reports, /reports/:id, /reports/:id/csv, /reports/:id/pdf	User
Live updates	GET /events (Server-Sent Events)	User

Table 5.1: Summary of the REST API

Data layer. The operational data lives in MongoDB. Users and devices use UUID identifiers and every other collection a small integer identifier produced by an atomic counter, so that the identifiers seen through the API remain short and stable. Indexes are created at start-up, including a unique index on the e-mail address of a user, on the name and the asset number of a device and on the hash of the device key, a compound index that serves the telemetry queries of a device, a time-to-live index that expires readings after the retention period (seven days by default), and a partial unique index that allows only one active model version per scope. MongoDB has no foreign-key cascade, so deleting a device removes its readings, alerts, firmware updates, maintenance records and predictions in application code and keeps its security incidents with the device link cleared. The knowledge base and the conversations are stored in PostgreSQL through versioned SQL migrations that are applied in order, recorded with a checksum and protected by an advisory lock. The platform was first built entirely on PostgreSQL and was later moved to the present division of labour; a migration tool copies an existing installation into MongoDB while preserving all identifiers. When it was run on the development installation it copied 18 devices, 100,614 readings, 1,408 alerts and 1,462 prediction snapshots with exactly matching counts, and running it a second time added nothing.

Security. Passwords are hashed with bcrypt and sessions use signed tokens. A device key is a random 128-bit value of which only the SHA-256 hash is stored. The API sets security headers through Helmet, limits the size of bodies, and applies Redis-backed rate limits (600 requests a minute per address for the API, 300 per device key, 10 failed sign-ins in 15 minutes, 10 uploads a minute, 20 reports and 20 assistant messages a minute per user). Webhook addresses on private, loopback or link-local networks are refused when saved and again on every delivery, redirects are not followed, and the payload is signed. CSV exports neutralise cells that start like a spreadsheet formula. Uploaded files are accepted only when both the extension and the content agree (the PDF header, the structure of a DOCX archive with a check that rejects archives that would expand to an unreasonable size, valid UTF-8 text without binary data), are stored under random names and are never executed. The signing secret of the tokens is read from the environment or generated once and kept in the data volume; no secret is stored in the source code.

Prediction. The predictor is described in detail in Section 5.2. The model supplied with the system was trained on 8,000 synthetic episodes of drift, leak and recovery expressed in normalised units, so that one model serves every signal, and was evaluated on 3,000 further episodes. Table 5.2 gives the result.

Measure	Value	Meaning
AUC	0.94	Ability to rank episodes that cross the limit above those that do not
Precision	89 %	Share of raised warnings that were followed by a crossing
Recall	73 %	Share of crossings that were warned in advance
F1 score	0.80	Harmonic mean of precision and recall
Accuracy	92 %	Share of all windows classified correctly
Brier score	0.065	Mean squared error of the probabilities (lower is better)
Average lead time	7.6 readings	How far in advance a crossing was warned, on average

Table 5.2: Evaluation of the built-in model on synthetic episodes

These figures describe how well the model recognises the patterns of the **synthetic generator**; they are not a claim about real machines, and the user interface labels the model's data source as synthetic. For real behaviour the platform can retrain a model for one machine type from the readings that type has actually produced. The retraining uses a time-ordered split (the older 80 percent of windows for training and the newest 20 percent for testing), removes the windows around the boundary whose look-ahead would overlap, and activates the new model only if its AUC is at least 0.70 and not clearly worse than that of the model it

would replace. A rejected model is still stored, with the reason, so that the decision can be reviewed. The gate is covered by the backend tests, which train models on controlled data and check that a poor model is stored but not activated.

Retrieval and the assistant. The embedding model produces 1024-dimension vectors, documents are split into passages of about 1000 characters with an overlap of 150, and a passage is returned only if its cosine similarity is at least 0.40 and within 0.20 of the best match, or if it contains the query words. These values were not guessed: unrelated questions were measured to score between 0.30 and 0.35 against every document, whereas genuinely related questions scored between 0.45 and 0.80, so the gate sits between the two groups. Questions that are answered completely by live data do not retrieve documents, and a question with no relevant source receives a fixed answer without calling the model. Citations produced by the model are verified; a citation that does not correspond to a supplied source is removed and reported.

Testing. Quality was checked at four levels. The results of the last complete run are given in Table 5.3. The backend tests run against real MongoDB, PostgreSQL with pgvector and Redis, in empty test databases that are recreated for each run. The assistant tests use a clearly labelled deterministic stand-in for the language model, because the continuous-integration environment has no model; this makes it possible to verify retrieval, access control, citations and the handling of failures, but it says nothing about the quality of real answers. For that reason three further tests were run separately against a real local model, and the observed behaviour is reported below.

Test level	Scope	Result
Backend unit and API	Model mathematics, authentication, devices, alerts, rules, retraining, webhooks, push updates, rate limits, document parsing and validation, chunking, retrieval and access control, assistant, reports	176 passed, 3 skipped (live-model tests are opt-in)
Live model tests	A grounded and cited answer, a planted instruction that must not be obeyed, an off-topic question	3 passed with phi4-mini and qwen3-embedding
Frontend component tests	Formatting, push-stream parser, sign-in, rules screen, safe Markdown rendering, streamed answers with citations, retry and offline states, upload validation, reports page	39 passed
End-to-end (Playwright, Chrome)	Thirteen-step story: register, device, telemetry, maintenance, deactivate and reactivate, alert, prediction, document upload, answer with citation, live-data answer, privacy between users, CSV and PDF reports, audit log	13 passed

Test level	Scope	Result
Static checks	TypeScript type check and oxlint on both packages	No errors, no warnings

Table 5.3: Results of the automated tests (run of 19 October 2026)

The tests found real defects that were then fixed, for example history endpoints that omitted three of the eight signals, a dashboard cache that served stale counts after a device was deactivated, label leakage at the boundary between training and test windows in the first version of the retraining code, a regular expression whose escape sequence was lost inside a template string, and web-server settings that would have rejected document uploads and buffered streamed answers.

Packaging and operation. The whole system starts with docker compose up. Five containers are used: MongoDB, PostgreSQL with pgvector (built from a small Dockerfile), Redis, the API and the Nginx front end; an optional sixth container can run Ollama. The database ports are bound to the local machine only. The start-up of the complete stack was verified with the migrated data, including a document upload of about 800 kilobytes and report downloads through Nginx.

Observed limitations. The default chat model (qwen2.5:7b) needs more graphics memory than the six gigabytes of the development laptop provide together with the desktop, so the smaller phi4-mini model was used; the model name is a setting. A language model running on the host is not reachable from a container when Ollama listens only on the loopback address, so the assistant is used with the API running directly on the host, or Ollama must be exposed deliberately. The quality of answers depends on the chosen model; the system limits the damage that a weak answer can do (sources are shown, unsupported citations are removed, nothing can be modified) but does not make the model smarter. Finally, the published accuracy is for synthetic data, as explained above. The screenshots of the running system are included at the end of this report.

5.2 KEY FUNCTIONS (CODE EXPLANATION)

This section explains the functions on which the behaviour of the platform depends. The code shown is taken from the source files of the project.

Telemetry ingestion (services/ingest.ts). The function ingest is the single entry point for every reading, whether it comes from a real device or from the simulator. It stores the reading, refreshes the hot state in Redis, sets the heartbeat key with an expiry equal to the

offline limit, clears any heartbeat alert of a device that is reporting again, evaluates the rules of the device's machine type, runs the predictor, raises or clears predictive alerts, records the prediction and finally updates the device status.

```
export async function ingest(deviceId: string, input: MetricInput = {}, ts = new
Date()) {
  const m = emptyMetrics();
  for (const k of METRIC_KEYS) m[k] = input[k] ?? null;
  const hasMetrics = METRIC_KEYS.some((k) => m[k] !== null);
  const type = await deviceTypeOf(deviceId);

  if (hasMetrics) {
    await col('telemetry').insertOne({ deviceId, ts, ...m });
    const point = JSON.stringify({ ts: ts.getTime(), ...m });
    await redis
      .multi()
      .lpush(keys.recent(deviceId), point)
      .ltrim(keys.recent(deviceId), 0, RECENT_MAX - 1)
      .set(keys.latest(deviceId), point)
      .exec();
  }
  await redis.set(keys.hb(deviceId), '1', 'PX', config.offlineAfterMs);

  // a returning device clears its heartbeat alert
  await clearAlerts(deviceId, 'heartbeat');

  let degraded = false;

  // ---- rule-based threshold alerts ----
  if (hasMetrics) {
    const rules = await rulesForType(type);
    for (const key of METRIC_KEYS) {
      const v = m[key];
      if (v === null) continue;
      const { label, unit } = METRICS[key];
      const hit = rules.filter((r) => r.enabled && r.metric === key && (r.operator
=== '>' ? v > r.threshold : v < r.threshold));
      const worst = hit.find((r) => r.severity === 'critical') ?? hit[0];
      if (worst) {
        degraded = true;
        await raiseAlert({
          deviceId, type: 'threshold', severity: worst.severity, metric: key,
value: v,
          message: `${label} ${v.toFixed(1)}${unit} is ${worst.operator === '>' ?
'above' : 'below'} the ${worst.severity} limit (${worst.threshold}${unit})`,
        });
      } else {
        await clearAlerts(deviceId, 'threshold', key);
      }
    }

    // ---- predictive alerts: warn BEFORE the threshold is crossed ----
    const { prediction } = await predictDevice(deviceId, type);
    recordPredictions(deviceId, prediction.metrics).catch((e) =>
console.error('[prediction-history]', e.message));
    for (const f of prediction.metrics) {
```

```
      if (f.breached) {
        await clearAlerts(deviceId, 'predictive', f.metric);
      } else if (f.probability >= ALERT_PROB) {
        await raiseAlert({
          deviceId, type: 'predictive', severity: 'warning', metric: f.metric,
value: f.current,
          message: `${f.label} trending to ${f.threshold}${f.unit.trim() === '%' ?
'%' : f.unit}${f.secondsToBreach ? ` in ~${fmtEta(f.secondsToBreach)}` : ''}
(model confidence ${Math.round(f.probability * 100)}%)`,
        });
      } else if (f.probability < 0.3) {
        await clearAlerts(deviceId, 'predictive', f.metric);
      }
    }
  }

  await col('devices').updateOne({ _id: deviceId as any }, { $set: { lastSeen: ts,
status: degraded ? 'degraded' : 'online' } });
  publishTick();
}
```

Listing 5.1: ingest() - the pipeline applied to every reading

The order matters: the reading is stored first, so that it is never lost if a later step fails, and the rule evaluation precedes the prediction, so that a signal that has already breached is reported by the threshold alert and not predicted again. Alerts are refreshed and not duplicated, because raiseAlert looks for an unresolved alert with the same device, type and metric before it creates a new one.

Feature extraction (ml/features.ts). The predictor looks at the last twenty readings of one signal. A straight line is fitted by least squares, which removes the noise and gives the level, the slope and the residual spread. The function then expresses the situation in five numbers that are independent of the unit of the signal: the headroom to the threshold, the movement expected over the horizon, the margin left at the end of the horizon, the volatility and the ratio of trend to headroom. Each number is clipped to a sensible range so that a single odd reading cannot dominate.

```
export function extract(stress: number[], T: number, R: number): { x: number[];
fit: LinFit } | null {
  if (stress.length < MIN_POINTS) return null;
  const fit = linFit(stress.slice(-WINDOW));
  const h = (T - fit.last) / R; // headroom
  const g = (fit.slope * HORIZON) / R; // movement expected over the horizon
  return {
    fit,
    x: [
      clip(h, -0.5, 1),
      clip(g, -0.5, 0.8),
      clip(h - g, -1, 1.5),
      clip(fit.resid / R, 0, 0.2),
      clip(g / Math.max(h, 0.03), -1, 5),
```

```
    ],
  };
}
```

Listing 5.2: extract() - turning a window of readings into model features

Scoring and explanation (ml/model.ts). The model is a logistic regression. The features are standardised with the means and standard deviations stored in the model, multiplied by the weights, added to the bias, and passed through the logistic function. Because the model is linear, the contribution of each feature to the log-odds is the product of its weight and its standardised value; these contributions and the bias add up exactly to the log-odds, which is what the explanation panel of the interface displays.

```
export function scoreFeatures(model: Pick<TrainedModel, 'weights' | 'bias' |
'mean' | 'std'>, x: number[]): number {
  let s = model.bias;
  for (let i = 0; i < x.length; i++) s += (model.weights[i] * (x[i] -
model.mean[i])) / model.std[i];
  return sigmoid(s);
}
```

Listing 5.3: scoreFeatures() - the logistic-regression score

The validation gate of retraining (services/retrain.ts). After a model has been fitted on the older windows and evaluated on the newest ones, the decision whether it may replace the active model is a single expression, followed by a human-readable reason that is stored with the version whether or not the model was accepted.

```
  const activated = evaluation.auc >= MIN_AUC && evaluation.auc >= previous.auc -
0.03;
  const reason = activated
    ? `Activated: AUC ${evaluation.auc.toFixed(2)} on ${test.length} recent held-
out windows (previous model ${previous.auc.toFixed(2)}).`
    : evaluation.auc < MIN_AUC
      ? `Kept the previous model: the new one only reached AUC
${evaluation.auc.toFixed(2)} (needs ${MIN_AUC}).`
      : `Kept the previous model: it scores AUC ${previous.auc.toFixed(2)} on the
same data, higher than the new ${evaluation.auc.toFixed(2)}.`;
```

Listing 5.4: the rule that decides whether a retrained model is activated

Heartbeat sweeper (services/sweeper.ts). Offline detection does not need a periodic comparison of timestamps in the database. Each reading sets a Redis key with a time-to-live; when the key disappears the device has been silent for longer than the limit. The sweeper only has to look for devices that are marked as online but whose key no longer exists, mark them offline and raise a critical alert.

One access rule for all documents (services/kb/store.ts). The most important security decision of the knowledge base is a single function that builds the condition used by

listing, searching, downloading and the retrieval performed for the assistant. An administrator may read every document; any other user may read the documents they own and the documents that were shared. Because the condition is part of the SQL query that ranks the passages, the text of a document that the user may not read is never selected and therefore can never reach the prompt.

```
export function accessPredicate(user: Viewer, params: unknown[], alias = 'd'):
string {
  if (user.role === 'admin') return 'TRUE';
  params.push(user.id);
  return `(${alias}.owner_id = $${params.length} OR ${alias}.visibility =
'shared')`;
}
```

Listing 5.5: accessPredicate() - who may read which document

Hybrid retrieval (services/kb/store.ts). The search runs two rankings, a nearest-neighbour search on the embeddings and a PostgreSQL full-text search, merges them by reciprocal rank fusion (each ranking contributes 1 / (60 + rank)), and then applies the relevance gate described in Section 5.1.

```
    // relevance gate: semantically close enough (absolutely, and relative to the
best match) or an actual keyword match
    const best = Math.max(0, ...[...byId.values()].map((h) => h.vectorScore ??
0));
    const floor = Math.max(minScore, best - config.ragRelativeMargin);
    const merged = [...byId.values()]
      .filter((h) => (h.vectorScore !== null && h.vectorScore >= floor) ||
(h.keywordRank !== null && h.keywordRank > 0))
      .map((h) => ({
        ...h,
        score: ((h as any).vRank !== undefined ? 1 / (RRF_K + (h as any).vRank +
1) : 0) + ((h as any).kRank !== undefined ? 1 / (RRF_K + (h as any).kRank + 1) :
0),
      }))
      .sort((a, b) => b.score - a.score);
```

Listing 5.6: relevance gate and reciprocal rank fusion

Treating documents as data (services/ai/rag.ts). Text that comes from a document or from the database is untrusted. Before it is placed between the source markers of the prompt, control characters are removed, text that imitates the markers is replaced, and text that imitates a citation is changed so that a document cannot invent a source.

Verifying citations (services/ai/rag.ts). After the model has finished, finalizeAnswer compares every citation in the answer with the identifiers of the sources that were actually supplied. Unknown identifiers are removed and reported, the sources that were really cited

are marked, and an answer that cites nothing is flagged, so that the interface can show which statements are backed by evidence.

```
export function finalizeAnswer(text: string, sources: Source[]) {
  const valid = new Set(sources.map((s) => s.id));
  const cited: string[] = [];
  const invalid: string[] = [];
  const cleaned = text.replace(SOURCE_RE, (_all, group: string) => {
    const ids = group.split(',').map((s) => s.trim());
    const keep = ids.filter((id) => { const ok = valid.has(id); (ok ? cited :
invalid).push(id); return ok; });
    return keep.map((id) => `[${id}]`).join('');
  }).replace(/[ \t]+\n/g, '\n').trim();
  const uniqueCited = [...new Set(cited)];
  return {
    text: cleaned,
    cited: uniqueCited,
    invalid: [...new Set(invalid)],
    uncited: sources.length > 0 && uniqueCited.length === 0,
    sources: sources.map((s) => ({ ...s, cited: uniqueCited.includes(s.id) })),
  };
}
```

Listing 5.7: finalizeAnswer() - checking the model's citations

Streaming one answer (routes/assistant.ts). The handler first prepares the turn (tools and retrieval), then switches the response to a server-sent event stream. A listener on the connection aborts the model call if the user presses stop or closes the page. When the stream ends, the final text is checked, saved together with its sources, tool calls, model and duration, and an audit entry that contains no text of the conversation is written.

Streaming a large report (services/reports.ts). A CSV export can contain hundreds of thousands of rows. The function writes the header, then reads the data in batches of 2,000 rows by paging on the key of the last row, writes every batch as it arrives, and waits for the connection to drain when the client is slow, so that memory use stays constant.

```
export async function streamCsv(def: ReportDef, f: Filters, viewer: Viewer, res:
Response, signal: { aborted: boolean }) {
  res.write('' + csvHeader(def.columns)); // BOM so Excel reads UTF-8 correctly
  let n = 0;
  for await (const rows of batches(def, f, viewer, config.reportMaxRows)) {
    if (signal.aborted) return n;
    n += rows.length;
    if (!res.write(csvBody(rows, def.columns))) await new Promise<void>((r) =>
res.once('drain', () => r()));
  }
  return n;
}
```

Listing 5.8: streamCsv() - exporting without loading everything into memory

CHAPTER 6

CONCLUSION & FUTURE WORK

This chapter deals with the outcome of the project and its future. It summarises what has been achieved against the objectives set in Chapter 1, states honestly what the work does not show, and proposes enhancements that would take the platform from a complete prototype towards a production system.

6.1 CONCLUSION

The project has produced a working platform that monitors the health and the security of IoT devices, predicts imminent breaches of alert limits with an explanation, and answers maintenance questions from live data and from the organisation's own documents. All the objectives stated in Chapter 1 were met: devices of different machine types are monitored with their own limits; readings are stored and analysed as they arrive; alerts, incidents, firmware status and maintenance history are available in one place and are pushed to the dashboard; every prediction is explained with exact feature contributions; models are retrained from telemetry, evaluated on newer data, versioned and protected by a validation gate; documents are uploaded, validated and indexed; and the assistant gives grounded answers with verifiable citations while being unable to change anything.

The design shows that a language model can be added to an operational system responsibly. The model is only the last step of a pipeline whose earlier steps are deterministic: a planner selects fixed read-only queries, the retrieval applies the user's access rights inside the database query, the text of documents is treated as data, and the answer is checked before it is stored. Running the model locally removes the need to send maintenance documents to an external service. Splitting the data between MongoDB (operational documents and time-stamped readings), PostgreSQL with pgvector (vectors, full-text search and access control in one query) and Redis (hot state and messaging) gave each kind of data a suitable home, and the migration tool shows that an installation can be moved between stores without loss.

The quality of the result is supported by evidence: 176 backend tests, 39 front-end tests and a thirteen-step browser test passed, the type checker and the linter report nothing, the whole stack starts from a single command, and several real defects were found and corrected through testing. At the same time the work has limits that should be stated plainly.

The accuracy of the supplied model was measured on synthetic data and therefore does not show how it would perform on real machines; the labels mean that a threshold was crossed, not that equipment failed; the assistant was verified with a small local model, and the best default model needs more graphics memory than the development laptop has; the continuous-integration workflow had not yet run on the hosting service when this report was written; and the system has not been tested with a large fleet of real devices or under heavy load.

6.2 FUTURE ENHANCEMENTS

Several improvements would increase the value and the robustness of the platform. The most important is **validation with real devices**: connecting physical equipment, collecting a period of genuine data, labelling real failures, retraining the models and measuring their accuracy on real behaviour. This would also make it possible to compare the logistic-regression model with other models, such as gradient-boosted trees or sequence models, while keeping the explanation requirement.

On the device side, gateways for the protocols that are common in industry (MQTT, OPC UA and CoAP) can be added in front of the REST API, together with signed firmware images and a real over-the-air update mechanism instead of the simulated roll-out. On the security side, HTTPS termination with certificate management, single sign-on, fine-grained permissions in addition to the two roles, secret rotation, and e-mail, SMS or messaging-application notifications would be needed in production.

For the assistant, optical character recognition would allow scanned manuals to be indexed; a formal evaluation set of questions with known answers would allow the retrieval settings and the choice of models to be tuned and compared objectively; larger or fine-tuned models could be used where hardware permits; multilingual documents and questions could be supported; and an approval workflow could let the assistant propose actions (for example creating a maintenance entry) that a person confirms, while keeping the rule that the assistant itself never writes.

On the data side, a MongoDB replica set would enable multi-document transactions and high availability, long-term aggregated telemetry would allow trends over months while the raw readings expire, and monitoring of the models for drift would signal when retraining is due. Finally, container orchestration, centralised logging and a mobile application for engineers in the field would suit larger deployments.

APPENDIX

SOURCE CODE:

1. Project Structure

Project Folder Structure

```
iot-sentinel/
|-- backend/
|   |-- migrations/          versioned SQL for PostgreSQL (knowledge base, chats)
|   |-- src/
|   |   |-- config.ts, app.ts, index.ts, auth.ts
|   |   |-- mongo.ts         MongoDB connection, ids, indexes
|   |   |-- db.ts            PostgreSQL pool and migration runner
|   |   |-- redis.ts         Redis client and caches
|   |   |-- routes/          auth, users, devices, telemetry, alerts, rules,
anomaly,
|   |   |                    firmware, incidents, webhooks, audit, knowledge,
|   |   |                    assistant, reports, simulator ...
|   |   |-- services/        ingest, alerts, rules, prediction, retrain, sweeper,
|   |   |   |                seed, simulator, webhooks, reports ...
|   |   |   |-- ai/          Ollama provider, planner, tools, RAG, chat store
|   |   |   `-- kb/          extract, chunk, ingest queue, hybrid search
|   |   |-- ml/              features, logistic regression, predictor
|   |   `-- scripts/         importFromPostgres.ts
|   |-- tests/               unit, api, live (opt-in)
|   `-- Dockerfile, package.json
|-- frontend/
|   |-- src/pages/           Dashboard, Devices, Alerts, Anomaly, Reports,
|   |                        Assistant, KnowledgeBase, AuditLog, ...
|   |-- src/components/      charts, explanation panel, Markdown renderer
|   |-- src/api.ts, live.ts  REST/streaming client, Server-Sent Events client
|   `-- tests/, nginx.conf, Dockerfile, package.json
|-- e2e/                     Playwright end-to-end scenario
|-- docker/postgres/         PostgreSQL 16 + pgvector image
|-- docs/                    architecture, API, ML, RAG, database, deployment ...
`-- docker-compose.yml, .env.example, README.md
```

2. Backend Package Installation

```
# backend (Node.js + Express + TypeScript)
mkdir iot-sentinel && cd iot-sentinel
mkdir backend && cd backend
npm init -y
npm install express cors helmet jsonwebtoken bcryptjs zod dotenv multer
npm install mongodb pg ioredis
npm install pdfjs-dist mammoth csv-parse pdfkit
npm install -D typescript tsx @types/node @types/express @types/cors
npm install -D @types/jsonwebtoken @types/multer @types/pg @types/pdfkit
npm install -D vitest supertest @types/supertest pdf-lib oxlint

# infrastructure (MongoDB, PostgreSQL + pgvector, Redis)
docker compose up -d mongo postgres redis

# run
npm run dev                      # API on http://localhost:4000
```

```
# frontend (React + TypeScript + Vite)
cd .. && npm create vite@latest frontend -- --template react-ts
cd frontend
npm install react-router-dom @tanstack/react-query recharts lucide-react
npm install -D vitest jsdom @testing-library/react @testing-library/user-event
@testing-library/jest-dom oxlint
npm run dev                      # UI on http://localhost:5173 (proxies /api)

# tests
cd ../backend && npm test
cd ../frontend && npm test
cd ../e2e && npm install && npx playwright test
```

3. Database Connection

MongoDB connection (backend/src/mongo.ts)

```
export const mongo = new MongoClient(config.mongoUrl, { maxPoolSize: 20 });
export const mdb: Db = mongo.db();

export type Doc = any;
export const col = (name: string): Collection<any> => mdb.collection<any>(name);
```

Atomic integer ids (backend/src/mongo.ts)

```
export async function nextId(name: string): Promise<number> {
  const r = await col('counters').findOneAndUpdate({ _id: name as any }, { $inc: {
seq: 1 } }, { upsert: true, returnDocument: 'after' });
  return (r as Doc).seq as number;
}
```

PostgreSQL connection for the knowledge base (backend/src/db.ts)

```
export const pool = new Pool({ connectionString: config.databaseUrl });

/** Run a query and return camelCased rows. */
export async function q<T = any>(text: string, params: unknown[] = []):
Promise<T[]> {
  const res = await pool.query(text, params as any[]);
  return camelize<T[]>(res.rows);
}

export async function q1<T = any>(text: string, params: unknown[] = []): Promise<T
| undefined> {
  return (await q<T>(text, params))[0];
}
```

4. Environment Variables

.env.example

```
# Copy to .env and edit. Docker Compose reads it automatically.

# A long random string used to sign sign-in tokens. Generate one with:  openssl
rand -hex 32
```

```
JWT_SECRET=change-me-in-production

# Password for the Postgres container (also used by the backend). Changing it
after the first run
# requires resetting the volume:  docker compose down -v
POSTGRES_PASSWORD=iot

# Password for the MongoDB container (operational data). Changing it after the
first run requires resetting its volume.
MONGO_PASSWORD=iot

# Turn the built-in device simulator off when only real devices report:
SIMULATOR=true

# Origin allowed to call the API from a browser (the dashboard URL)
CORS_ORIGIN=http://localhost:8080
```

5. User Model

Role and authenticated user (backend/src/auth.ts)

```
export type Role = 'admin' | 'viewer';
export interface AuthUser { id: string; email: string; name: string; role: Role }
```

Input validation for registration (backend/src/routes/auth.ts)

```
const email = z.string().trim().toLowerCase().email().max(120);
const password = z.string().min(8, 'Password must be at least 8
characters').max(100);
```

Stored user document (inside the register handler)

```
    await col('users').insertOne({ _id: user.id as any, email: user.email, name:
user.name, passwordHash: await hashPassword(b.password), role: user.role,
createdAt: new Date() });
```

6. Password Hashing and Registration

Password hashing and token signing (backend/src/auth.ts)

```
export const hashPassword = (pw: string) => bcrypt.hash(pw, 10);
export const checkPassword = (pw: string, hash: string) => bcrypt.compare(pw,
hash);
export const signToken = (u: AuthUser) => jwt.sign({ sub: u.id },
config.jwtSecret, { expiresIn: '12h' });
```

Registration controller (backend/src/routes/auth.ts)

```
authRouter.post('/register', asyncH(async (req, res) => {
  const b = z.object({ name: z.string().trim().min(2).max(60), email, password
}).parse(req.body);
  if (await col('users').findOne({ email: b.email }, { projection: { _id: 1 } }))
throw new HttpError(409, 'An account with that email already exists');
  const first = (await col('users').estimatedDocumentCount()) === 0;
  const user: AuthUser = { id: crypto.randomUUID(), email: b.email, name: b.name,
role: first ? 'admin' : 'viewer' };
```

```
  try {
    await col('users').insertOne({ _id: user.id as any, email: user.email, name:
user.name, passwordHash: await hashPassword(b.password), role: user.role,
createdAt: new Date() });
  } catch (e) {
    if ((e as { code?: number }).code === 11000) throw new HttpError(409, 'An
account with that email already exists'); // two sign-ups raced
    throw e;
  }
  audit({ userId: user.id, userEmail: user.email, ip: req.ip }, 'auth.register',
'user', user.id, { role: user.role });
  res.status(201).json(session(user));
}));
```

7. Login and JWT Generation

Login controller (backend/src/routes/auth.ts)

```
authRouter.post('/login', asyncH(async (req, res) => {
  const b = z.object({ email, password: z.string().min(1) }).parse(req.body);
  const key = await limitLogins(req.ip ?? 'unknown');
  const row = await col('users').findOne({ email: b.email });
  if (!row || !(await checkPassword(b.password, row.passwordHash))) {
    audit({ userEmail: b.email, ip: req.ip }, 'auth.login_failed', 'user',
row?._id ?? null);
    throw new HttpError(401, 'Incorrect email or password');
  }
  await redis.del(key);
  const user: AuthUser = { id: row._id as unknown as string, email: row.email,
name: row.name, role: row.role };
  audit({ userId: user.id, userEmail: user.email, ip: req.ip }, 'auth.login',
'user', user.id);
  res.json(session(user));
}));
```

8. Authentication Middleware

JWT middleware and role guards (backend/src/auth.ts)

```
/** Any signed-in user. Loads the user fresh so deleted accounts and role changes
apply immediately. */
export const authenticate: RequestHandler = async (req, _res, next) => {
  try {
    const header = req.header('authorization') ?? '';
    const token = header.startsWith('Bearer ') ? header.slice(7) : '';
    if (!token) throw new HttpError(401, 'Sign in required');
    let sub: string;
    try {
      sub = (jwt.verify(token, config.jwtSecret) as { sub: string }).sub;
    } catch {
      throw new HttpError(401, 'Session expired - sign in again');
    }
    const row = await col('users').findOne({ _id: sub as any });
    if (!row) throw new HttpError(401, 'Account no longer exists');
    req.user = { id: row._id as unknown as string, email: row.email, name:
row.name, role: row.role };
    next();
  } catch (e) {
```

```
    next(e);
  }
};

/** Viewers are read-only; only admins may change anything. */
export const adminWrites: RequestHandler = (req, _res, next) => {
  if (req.method !== 'GET' && req.user?.role !== 'admin') return next(new
HttpError(403, 'Read-only account - ask an admin to make this change'));
  next();
};

/** Admin-only guard for reads as well as writes, for screens that expose secrets
or user data. */
export const requireAdmin: RequestHandler = (req, _res, next) => {
  if (req.user?.role !== 'admin') return next(new HttpError(403, 'Administrator
access required'));
  next();
```

9. Create API

Validation schema (backend/src/routes/devices.ts)

```
const createSchema = z.object({
  name: z.string().trim().min(2).max(60),
  type: z.string().trim().min(2).max(40),
  location: z.string().trim().max(60).default('Unassigned'),
  ipAddress: z.string().trim().max(45).optional(),
  firmwareVersion: z.string().regex(/^\d+\.\d+\.\d+$/, 'Use semantic version, e.g.
1.2.0'),
  simulated: z.boolean().default(false),
});
```

Create device handler (backend/src/routes/devices.ts)

```
devicesRouter.post('/', asyncH(async (req, res) => {
  const b = createSchema.parse(req.body);
  if (!(await getDeviceType(b.type))) throw new HttpError(400, `Unknown machine
type "${b.type}" - create it under Machine Types first`);
  const apiKey = newDeviceKey();
  const id = crypto.randomUUID();
  try {
    await col('devices').insertOne({
      _id: id as any, assetNo: await nextId('assetNo'), name: b.name, type:
b.type, location: b.location, ipAddress: b.ipAddress ?? null,
      firmwareVersion: b.firmwareVersion, status: 'offline', active: true,
simulated: b.simulated, apiKeyHash: hashKey(apiKey), lastSeen: null, createdAt:
new Date(),
    });
  } catch (e) {
    if ((e as { code?: number }).code === 11000) throw new HttpError(409, `A
device named "${b.name}" already exists`);
    throw e;
  }
  const created = (await getDeviceRow(id))!;
  const row = { id };
  audit(ctxOf(req), 'device.create', 'device', row.id, { name: b.name, type:
b.type, simulated: b.simulated });
  await clearCache('cache:dashboard');
```

```
  publish('device', { id: row.id });
  // the raw key is returned exactly once - only its hash is stored
  res.status(201).json({ ...(await withLatest([created]))[0], apiKey });
}));
```

10. Read API

List and search devices (backend/src/routes/devices.ts)

```
devicesRouter.get('/', asyncH(async (req, res) => {
  const { status, q: search, active } = req.query as Record<string, string |
undefined>;
  const rows = await listDevices({ status, active: active === 'true' ? true :
active === 'false' ? false : undefined, search: search?.trim() || undefined });
  res.json(await withLatest(rows));
}));
```

Read one device (backend/src/routes/devices.ts)

```
devicesRouter.get('/:id', asyncH(async (req, res) => {
  const row = await getDeviceRow(String(req.params.id));
  if (!row) throw new HttpError(404, 'Device not found');
  res.json((await withLatest([row]))[0]);
}));
```

11. Update

Validation schema (backend/src/routes/devices.ts)

```
const updateSchema = z.object({
  name: z.string().trim().min(2).max(60).optional(),
  location: z.string().trim().min(1).max(60).optional(),
  ipAddress: z.string().trim().max(45).nullable().optional(),
  active: z.boolean().optional(),
});
```

Update device handler (backend/src/routes/devices.ts)

```
devicesRouter.patch('/:id', asyncH(async (req, res) => {
  const b = updateSchema.parse(req.body);
  const found = await col('devices').findOne({ _id: String(req.params.id) as any
}, { projection: { name: 1, active: 1 } });
  if (!found) throw new HttpError(404, 'Device not found');
  const current = { id: found._id as unknown as string, name: found.name as
string, active: found.active !== false };
  const set: Record<string, unknown> = {};
  if (b.name !== undefined) set.name = b.name;
  if (b.location !== undefined) set.location = b.location;
  if (b.ipAddress !== undefined) set.ipAddress = b.ipAddress;
  if (b.active !== undefined) set.active = b.active;
  if (Object.keys(set).length) {
    try { await col('devices').updateOne({ _id: current.id as any }, { $set: set
}); } catch (e) {
      if ((e as { code?: number }).code === 11000) throw new HttpError(409, `A
device named "${b.name}" already exists`);
      throw e;
    }
```

```
  }
  if (b.active !== undefined && b.active !== current.active) {
    if (!b.active) {
      await col('alerts').updateMany({ deviceId: current.id, status: { $ne:
'resolved' } }, { $set: { status: 'resolved', resolvedAt: new Date() } });
      await redis.del(keys.hb(current.id));
    }
    await recordMaintenance(current.id, 'activation', b.active ? 'Device
reactivated' : 'Device deactivated (taken out of service)', '', { id:
req.user?.id, name: req.user?.name });
    audit(ctxOf(req), b.active ? 'device.activate' : 'device.deactivate',
'device', current.id, { name: current.name });
    publish('alert');
  } else {
    audit(ctxOf(req), 'device.update', 'device', current.id, { changed:
Object.keys(b) });
  }
  forgetDevice(current.id);
  await clearCache('cache:dashboard');
  publish('device', { id: current.id });
  res.json((await withLatest([(await getDeviceRow(current.id))!]))[0]);
}));
```

12. Delete

Delete device handler (backend/src/routes/devices.ts)

```
devicesRouter.delete('/:id', asyncH(async (req, res) => {
  const id = String(req.params.id);
  const gone = await col('devices').findOne({ _id: id as any }, { projection: {
name: 1 } });
  if (!gone) throw new HttpError(404, 'Device not found');
  await deleteDeviceData(id);
  await redis.del(keys.hb(id), keys.latest(id), keys.recent(id));
  forgetDevice(id);
  audit(ctxOf(req), 'device.delete', 'device', id, { name: gone.name });
  await clearCache('cache:dashboard');
  publish('device', { id });
  res.status(204).end();
}));
```

Removing a device together with its data (backend/src/mongo.ts)

```
export async function deleteDeviceData(deviceId: string) {
  await Promise.all([
    col('telemetry').deleteMany({ deviceId }),
    col('alerts').deleteMany({ deviceId }),
    col('firmware_updates').deleteMany({ deviceId }),
    col('maintenance_records').deleteMany({ deviceId }),
    col('prediction_history').deleteMany({ deviceId }),
    col('security_incidents').updateMany({ deviceId }, { $set: { deviceId: null }
}),
    col('devices').deleteOne({ _id: deviceId as any }),
  ]);
}
```

13. Express Routes

Route mounting and middleware order (backend/src/app.ts)

```
  app.use('/api/auth', authRouter);
  app.use('/api/telemetry', rateLimit({ name: 'device', max:
config.deviceRequestsPerMinute, by: byDeviceKey }), telemetryRouter);
  // knowledge base: any signed-in user may upload PRIVATE documents; the router
enforces ownership and sharing rules itself
  app.use('/api/knowledge', authenticate, knowledgeRouter);
  app.use('/api/assistant', authenticate, assistantRouter); // read-only for every
role: it can answer questions but never change data
  app.use('/api/reports', authenticate, reportsRouter); // read-only for every
role
  app.use('/api', authenticate, adminWrites);

  app.use('/api/events', eventsRouter);
  app.use('/api/dashboard', dashboardRouter);
  app.use('/api/devices', devicesRouter);
  app.use('/api/device-types', deviceTypesRouter);
  app.use('/api/simulator', simulatorRouter);
  app.use('/api/alerts', alertsRouter);
  app.use('/api/firmware', firmwareRouter);
  app.use('/api/incidents', incidentsRouter);
  app.use('/api/anomaly', anomalyRouter);
  app.use('/api/rules', rulesRouter);
  app.use('/api/webhooks', webhooksRouter);
  app.use('/api/users', usersRouter);
  app.use('/api/audit', auditRouter);
```

14. Main Server File

Server start-up (backend/src/index.ts)

```
async function main() {
  await waitFor('mongodb', () => pingMongo());
  await waitFor('postgres', () => pool.query('SELECT 1'));
  await waitFor('redis', () => redis.ping());
  await ensureIndexes();
  await migrate();
  const newTypes = await ensureReferenceData();
  loadModel();
  await loadStoredModels();
  await ensureGlobalVersion();
  await seedIfEmpty(newTypes);

  startEventBus();
  const ingestion = startIngestionWorker();
  syncSystemDocs().then((r) => { if (r.added || r.updated || r.removed)
console.log('[kb] application docs synced', r); }).catch((e) => console.warn('[kb]
doc sync failed', e.message));
  const sweeper = startSweeper();
  const sim = config.simulator ? startSimulator() : null;
  const server = createApp().listen(config.port, () =>
    console.log(`[api] listening on :${config.port}  (simulator ${config.simulator
? 'on' : 'off'})`));

  const shutdown = () => {
    clearInterval(sweeper);
```

```
    if (ingestion) clearInterval(ingestion);
    if (sim) clearInterval(sim);
    void stopEventBus();
    server.close(() => Promise.all([pool.end(), mongo.close(),
redis.quit()]).finally(() => process.exit(0)));
    setTimeout(() => process.exit(0), 3000).unref(); // open SSE streams must not
block shutdown
  };
  process.on('SIGINT', shutdown);
  process.on('SIGTERM', shutdown);
}

main().catch((e) => { console.error(e); process.exit(1); });
```

15. React Login Component

Login page (frontend/src/pages/AuthPages.tsx)

```
export function LoginPage() {
  const { login } = useAuth();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const { submit, error, busy } = useSubmit(() => login(email, password));
  return (
    <AuthLayout title="Welcome back" sub="Sign in to your monitoring console.">
      <form onSubmit={submit} className="stack" style={{ gap: 14 }}>
        <label className="field">Email<input className="input" type="email"
autoComplete="email" required autoFocus value={email} onChange={(e) =>
setEmail(e.target.value)} /></label>
        <label className="field">Password<input className="input" type="password"
autoComplete="current-password" required value={password} onChange={(e) =>
setPassword(e.target.value)} /></label>
        {error && <div className="err" role="alert">{error}</div>}
        <button className="btn primary" style={{ justifyContent: 'center' }}
disabled={busy}>{busy ? 'Signing in…' : 'Sign in'}</button>
      </form>
      <p className="muted small">New here? <Link to="/register"
className="link">Create an account</Link></p>
    </AuthLayout>
  );
}
```

16. Admin Dashboard Logic

Fetching the dashboard data (frontend/src/pages/Dashboard.tsx)

```
export default function Dashboard() {
  const nav = useNavigate();
  const [type, setType] = useState('');
  const { data: types } = useLive<DeviceType[]>('/device-types', false);
  const { data: s, error } = useLive<Summary>(`/dashboard/summary${type ?
`?type=${encodeURIComponent(type)}` : ''}`, 3000);
  const { data: preds } = useLive<FleetPrediction[]>('/anomaly/predictions',
3000);
  if (error && !s) return <ErrorBox error={error} />;
  if (!s) return <div className="empty">Loading fleet…</div>;

  const fleet = (preds ?? []).filter((p) => !type || p.type === type);
```

```
  const levels = { low: 0, elevated: 0, high: 0, breached: 0 } as Record<string,
number>;
  for (const p of fleet) levels[p.level] = (levels[p.level] ?? 0) + 1;
  const alertTotal = s.alerts.reduce((n, a) => n + a.count, 0);
  const critical = s.alerts.filter((a) => a.severity === 'critical').reduce((n, a)
=> n + a.count, 0);
  const predictive = s.alerts.filter((a) => a.type === 'predictive').reduce((n, a)
=> n + a.count, 0);
  const incidents = s.incidents.reduce((n, a) => n + a.count, 0);
  const atRisk = fleet.filter((p) => p.level === 'high' || p.level ===
'elevated');
  const healthy = s.totals.all ? Math.round((s.totals.online / s.totals.all) *
100) : 0;
```

Polling hook that slows down while push updates are connected (frontend/src/api.ts)

```
export function useLive<T>(path: string, interval: number | false = 3000, enabled
= true) {
  return useQuery<T>({
    queryKey: [path],
    queryFn: () => api.get<T>(path),
    refetchInterval: interval === false ? false : () => (isPushConnected() ?
Math.max(interval * 6, 20000) : interval),
    enabled,
    placeholderData: (prev) => prev,
  });
}
```

17. REST API Request and Response Examples

Example 1: Sign in

```
POST /api/auth/login HTTP/1.1
Host: localhost:4000
Content-Type: application/json

{
  "email": "admin@iot-sentinel.demo",
  "password": "********"
}
```

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "token": "eyJhbGciOiJIUzI1NiIs...<JWT>",
  "user": {
    "id": "9916804d-475f-4a88-90f0-e0350af533b6",
    "email": "admin@iot-sentinel.demo",
    "name": "Siddharth S",
    "role": "admin"
  }
}
```

Example 2: Create a device

```
POST /api/devices HTTP/1.1
```

```
Host: localhost:4000
Authorization: Bearer <token>
Content-Type: application/json

{
  "name": "boiler-room-sensor-07",
  "type": "Smart Sensor",
  "location": "Boiler Room",
  "firmwareVersion": "1.3.0"
}
```

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": "0ebdf39f-2b71-4746-96b1-4717ecb78d9b",
  "assetNo": 19,
  "name": "boiler-room-sensor-07",
  "type": "Smart Sensor",
  "location": "Boiler Room",
  "ipAddress": null,
  "firmwareVersion": "1.3.0",
  "status": "offline",
  "active": true,
  "simulated": false,
  "lastSeen": null,
  "createdAt": "2026-10-19T18:38:35.619Z",
  "openAlerts": 0,
  "latestFirmware": "1.3.0",
  "outdated": false,
  "latest": null,
  "apiKey": "<shown once - 32 hex characters>"
}
```

Example 3: Send telemetry (device key)

```
POST /api/telemetry HTTP/1.1
Host: localhost:4000
x-api-key: <device key>
Content-Type: application/json

{
  "temperature": 31.4,
  "cpu": 22,
  "memory": 47,
  "battery": 88,
  "signal": -61
}
```

```
HTTP/1.1 202 Accepted
Content-Type: application/json

{
  "accepted": true
}
```

SCREENSHOTS OF THE PROJECT

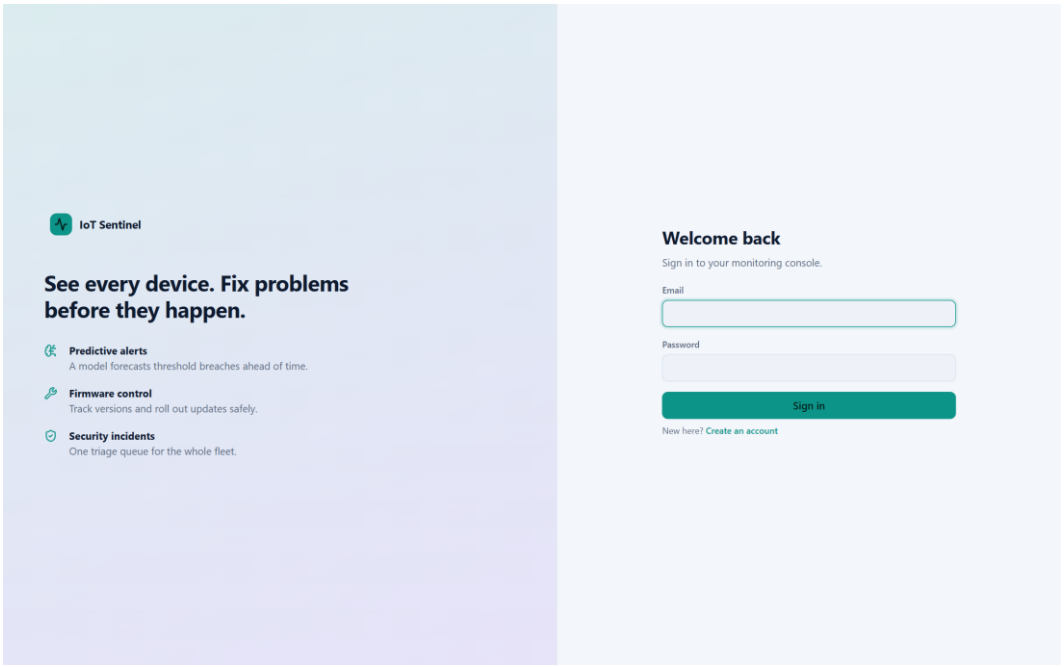


Figure 1: Sign-in screen

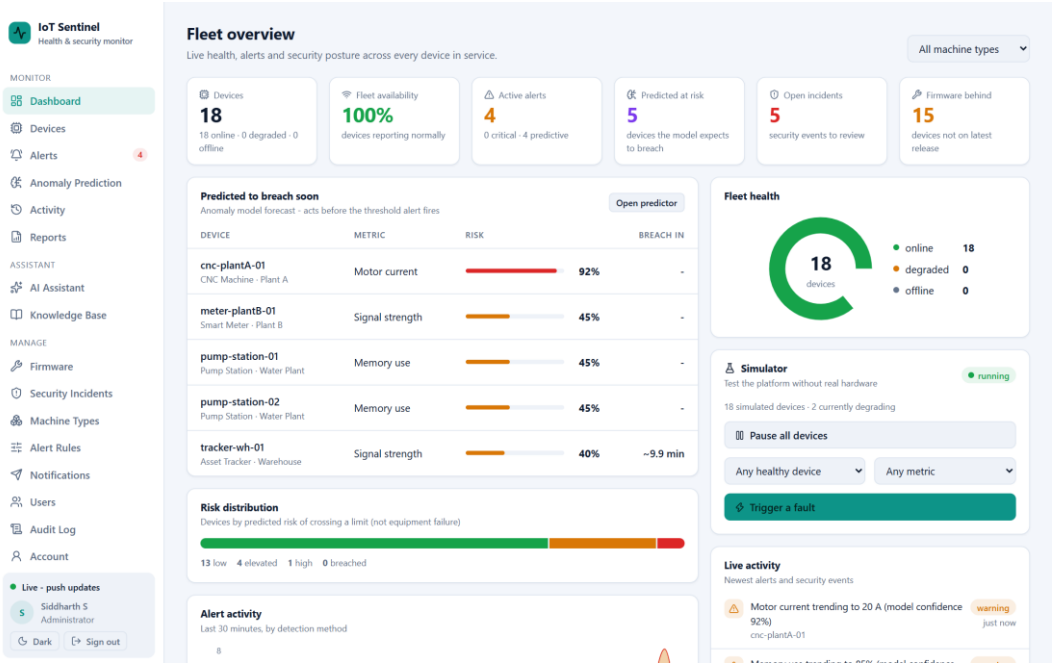


Figure 2: Fleet overview dashboard with live health, alerts and predicted risks

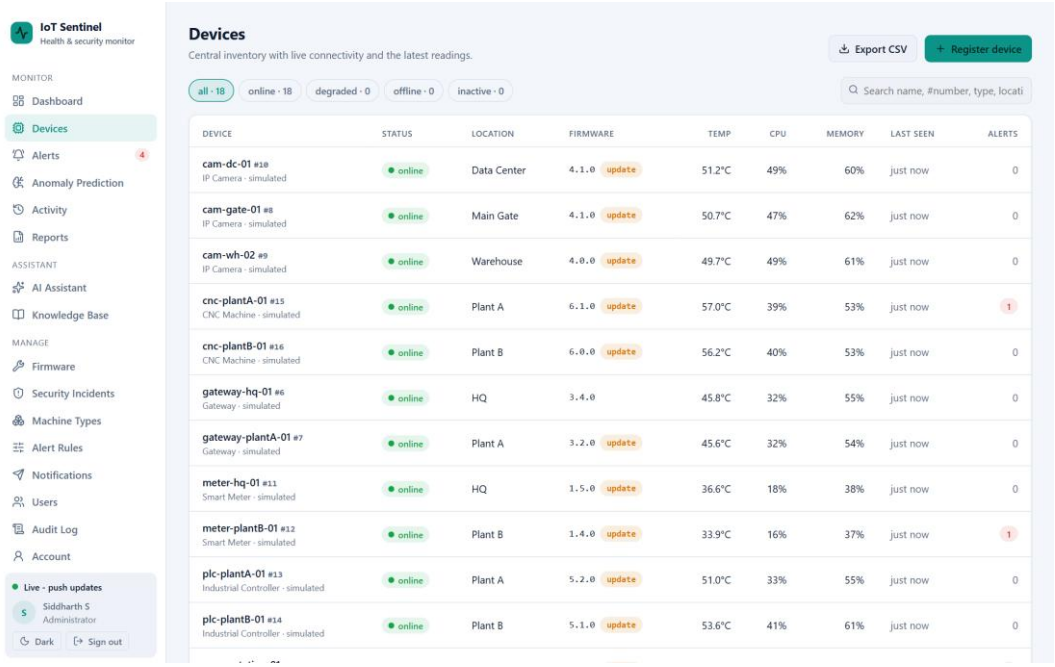


Figure 3: Device inventory with status, firmware and latest readings

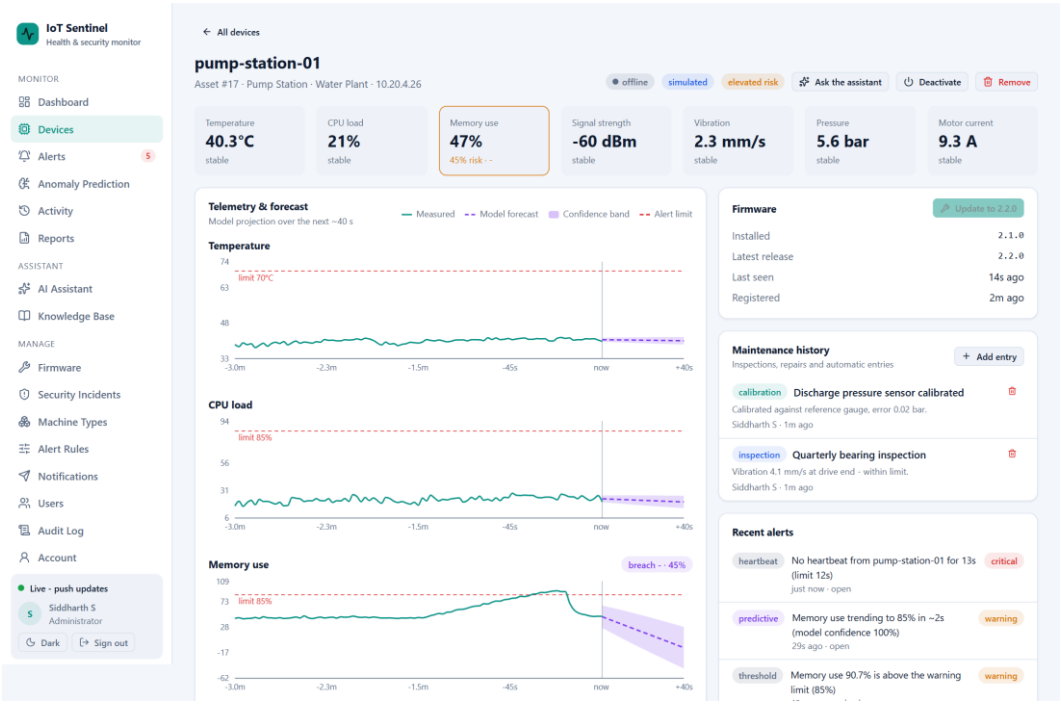


Figure 4: Device detail page with live readings and forecast charts (upper part of the page)

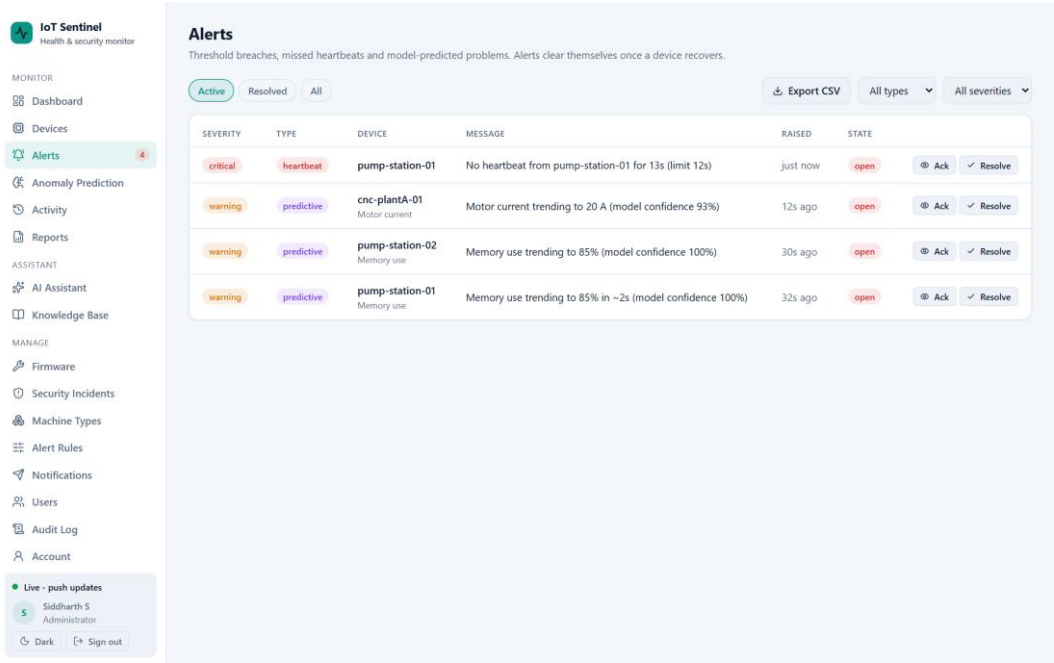


Figure 5: Alerts screen (threshold, heartbeat and predictive alerts)

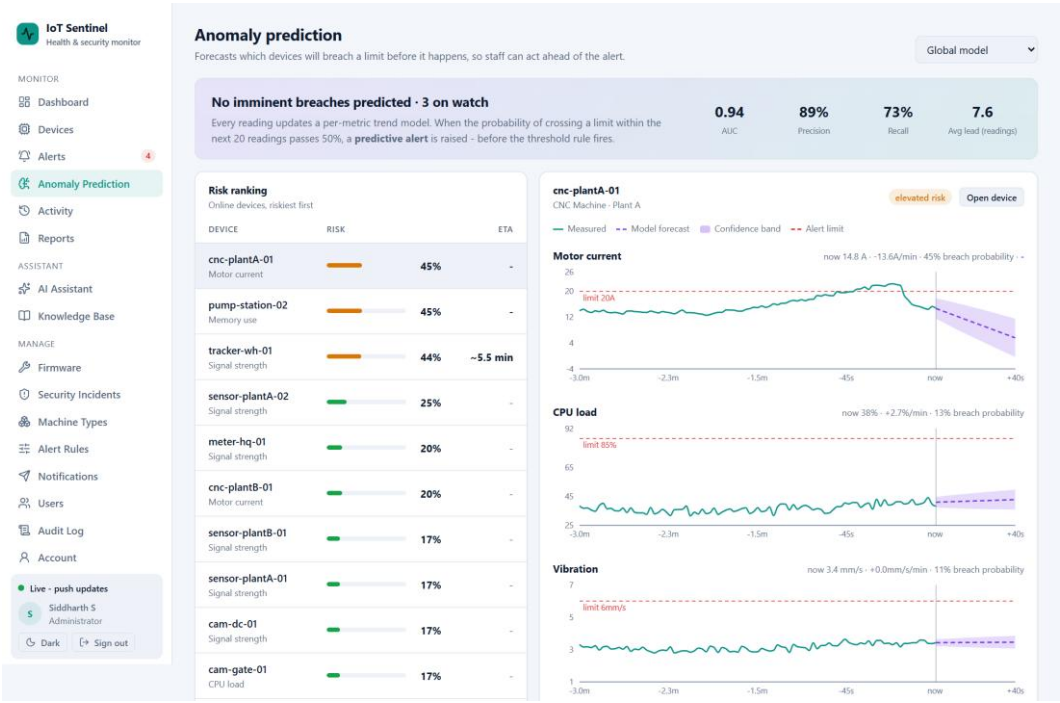


Figure 6: Anomaly prediction: fleet risk ranking and forecast chart with confidence band (upper part of the page)

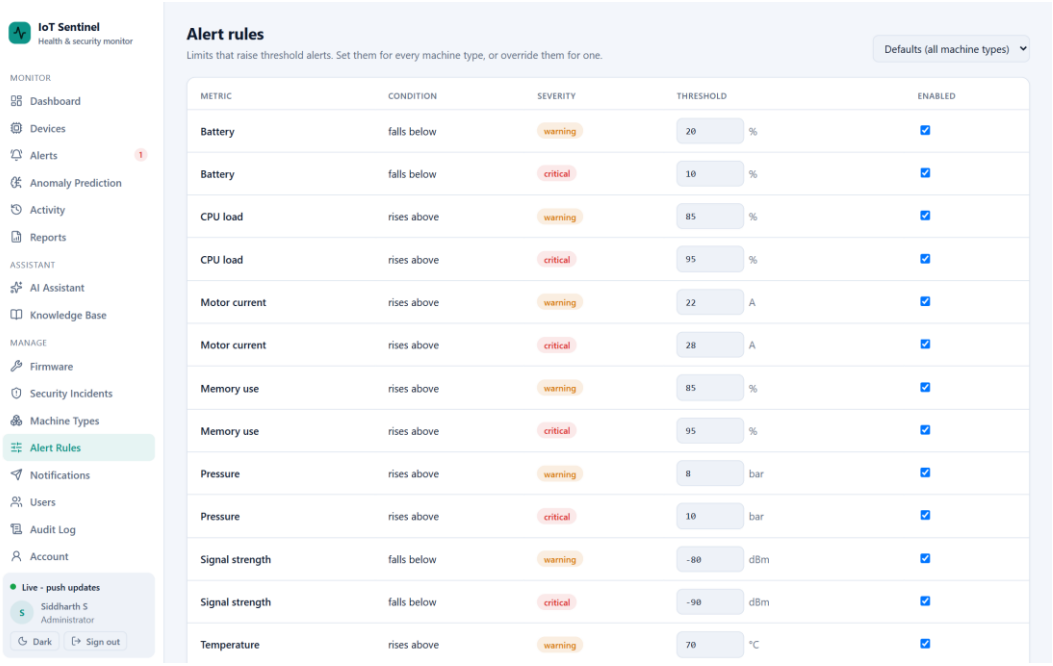


Figure 7: Alert rules with per-machine-type limits

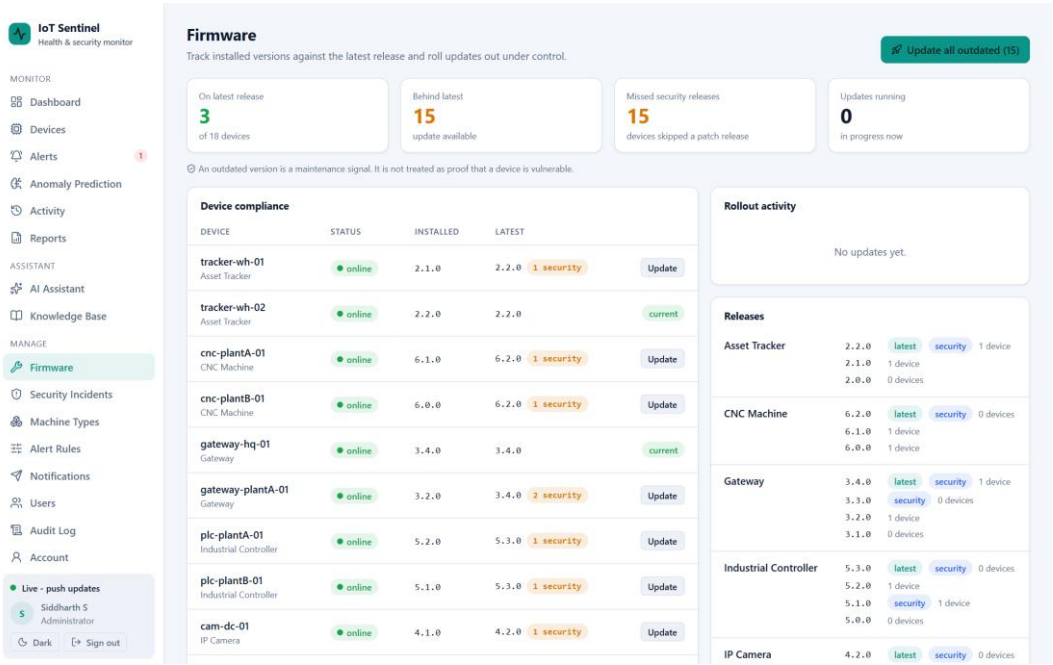


Figure 8: Firmware compliance and roll-outs

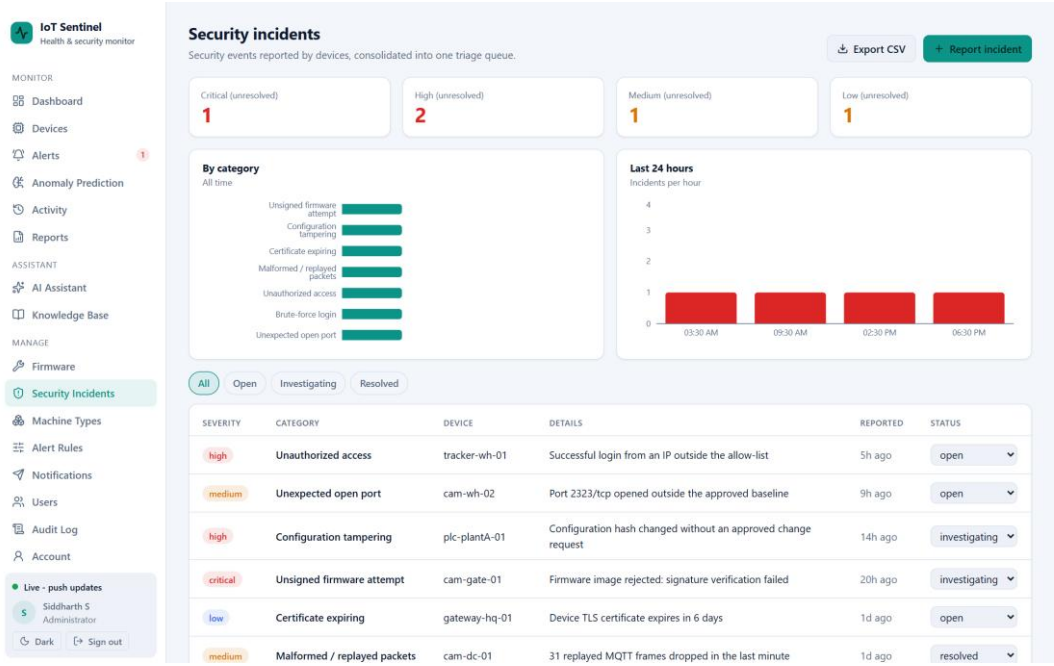


Figure 9: Security incident register

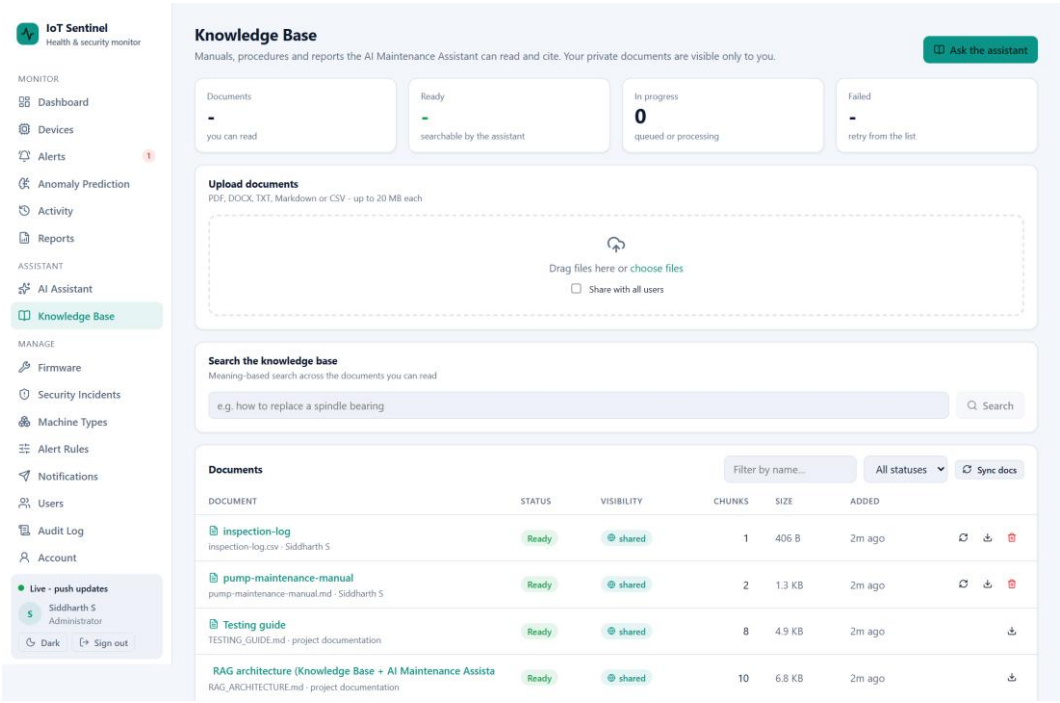


Figure 10: Knowledge base: upload and search (upper part of the page)

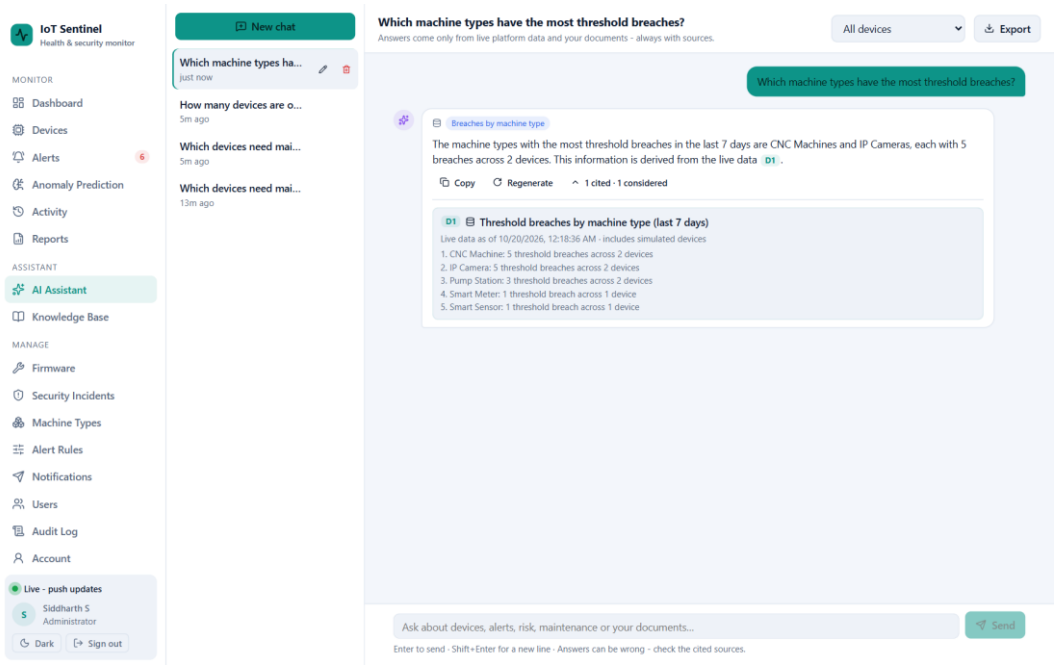


Figure 11: AI Maintenance Assistant answering from live data, with a citation and the tool used

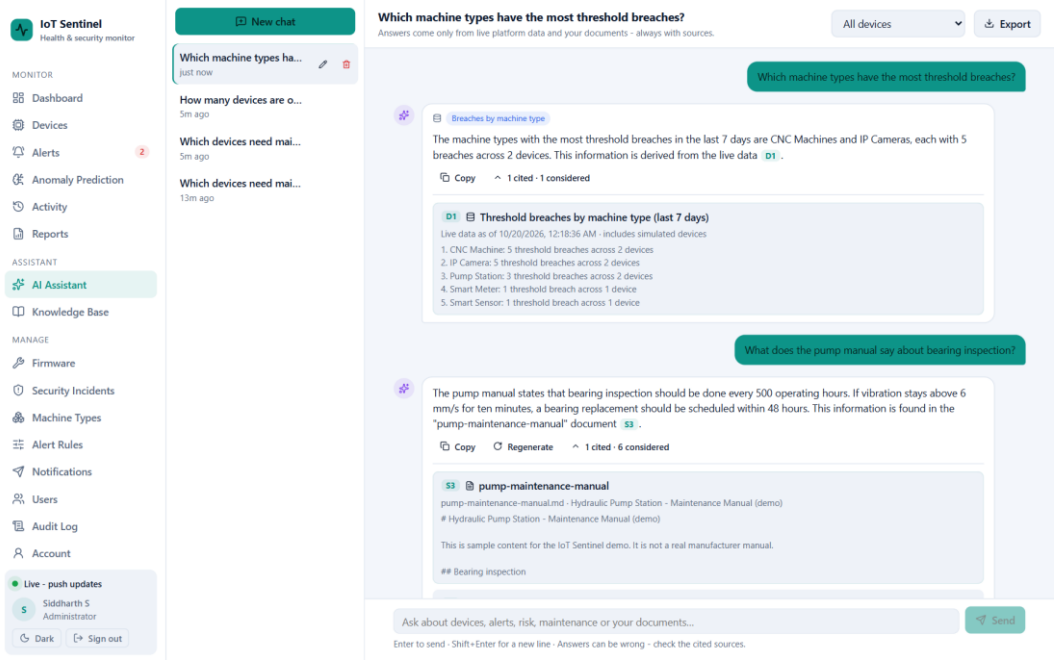


Figure 12: AI Maintenance Assistant answering from an uploaded manual, with the source shown

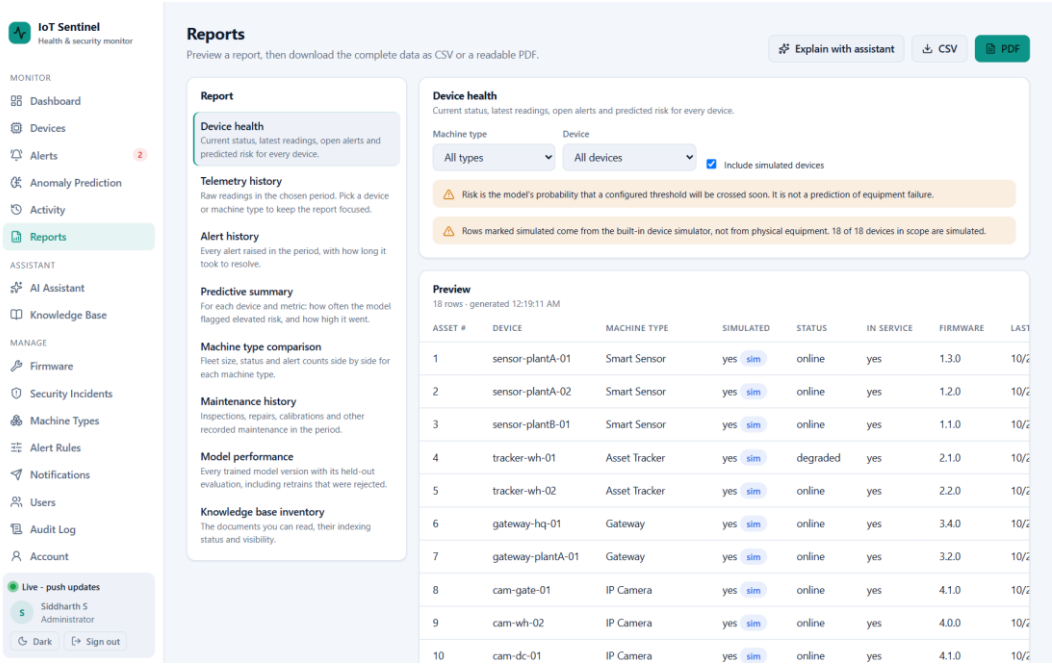


Figure 13: Reports: preview with filters and CSV / PDF download

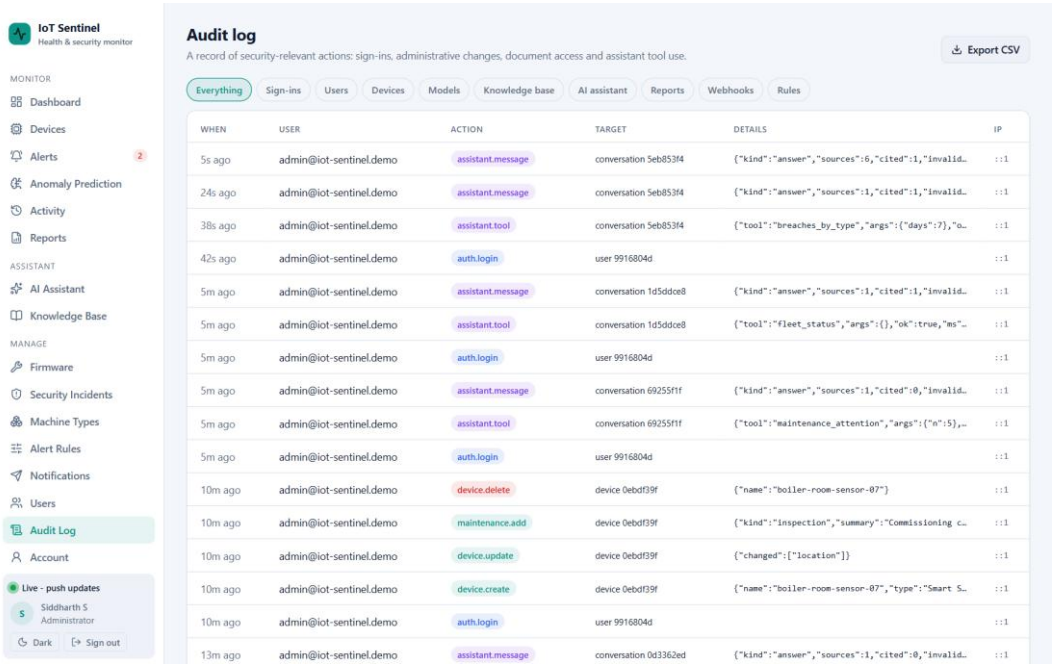


Figure 14: Audit log (administrators only)

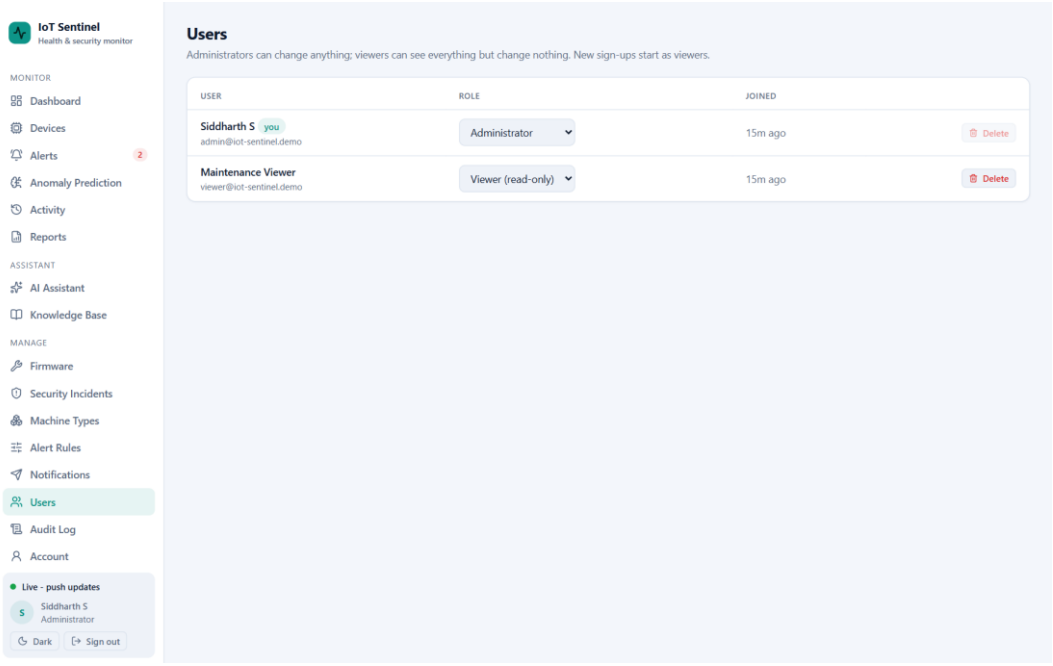


Figure 15: User management: the two roles, administrator and viewer (administrators only)

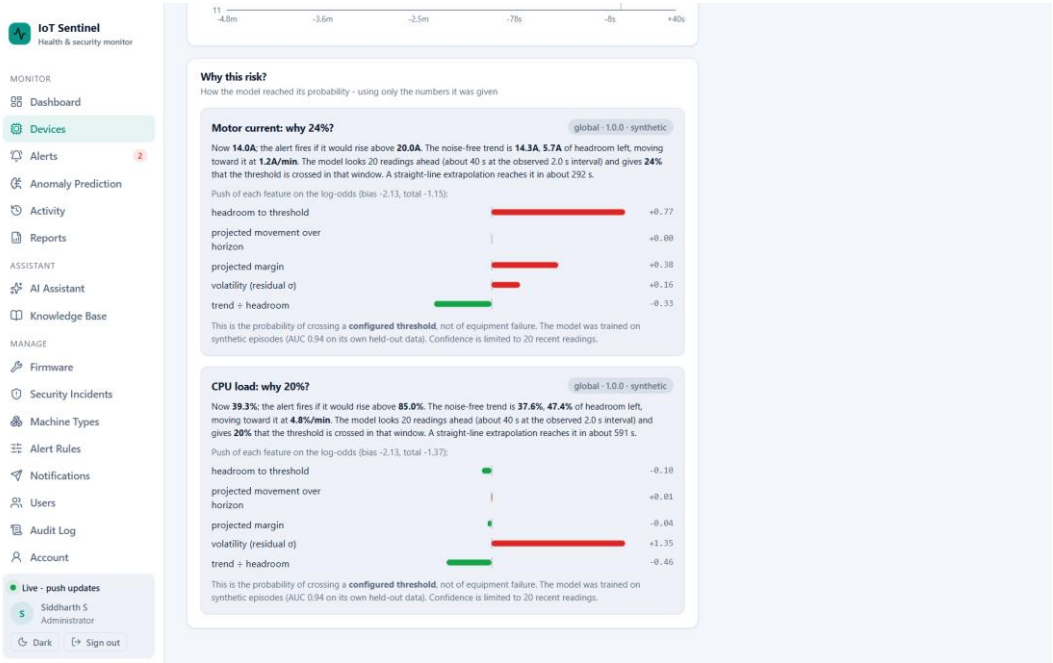


Figure 16: Why this risk? Explanation of a prediction for one device

REFERENCES

[1] J. Lee, B. Bagheri and H.-A. Kao, "A Cyber-Physical Systems architecture for Industry 4.0-based manufacturing systems," Manufacturing Letters, vol. 3, pp. 18-23, 2015.

[2] R. K. Mobley, An Introduction to Predictive Maintenance, 2nd ed. Amsterdam, The Netherlands: Butterworth-Heinemann, 2002.

[3] V. Chandola, A. Banerjee and V. Kumar, "Anomaly detection: A survey," ACM Computing Surveys, vol. 41, no. 3, article 15, pp. 1-58, 2009.

[4] M. Ahmed, A. N. Mahmood and J. Hu, "A survey of network anomaly detection techniques," Journal of Network and Computer Applications, vol. 60, pp. 19-31, 2016.

[5] D. W. Hosmer Jr., S. Lemeshow and R. X. Sturdivant, Applied Logistic Regression, 3rd ed. Hoboken, NJ, USA: Wiley, 2013.

[6] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, CA, USA, 2000.

[7] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015.

[8] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," https://owasp.org/www-project-top-10-for-large-language-model-applications/ (accessed Oct. 2026).

[9] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems 33 (NeurIPS 2020), 2020.

[10] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 42, no. 4, pp. 824-836, 2020.

[11] G. V. Cormack, C. L. A. Clarke and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in Proc. 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval, 2009, pp. 758-759.

[12] Qwen Team, "Qwen2.5 Technical Report," arXiv:2412.15115, 2024.

[13] A. Vaswani et al., "Attention is all you need," in Advances in Neural Information Processing Systems 30 (NIPS 2017), 2017.

[14] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec), 2023.

[15] A. Kane, "pgvector: Open-source vector similarity search for Postgres," https://github.com/pgvector/pgvector (accessed Oct. 2026).

[16] The PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," https://www.postgresql.org/docs/16/ (accessed Oct. 2026).

[17] MongoDB, Inc., "MongoDB Manual: Indexes, TTL indexes and the aggregation framework," https://www.mongodb.com/docs/manual/ (accessed Oct. 2026).

[18] Redis Ltd., "Redis documentation: Pub/Sub, key expiration and data types," https://redis.io/docs/ (accessed Oct. 2026).

[19] Ollama, "Ollama documentation and API reference," https://github.com/ollama/ollama/blob/main/docs/api.md (accessed Oct. 2026).